working html page code to make the scadual as calendar with time line indecator: PeriodTimeSubject/Class19:00 – 9:30Free / No Class29:30 – 10:00Free / No Class310:00 – 10:305A (NAFS M)410:30 – 11:00Free / No Class511:00 – 11:305B611:30 – 12:004A712:00 – 12:257A812:25 – 12:50Free / No Class

Building a schedule with a real-time indicator is a great way to stay on track. To make this work, we'll use **HTML** for the structure, **CSS** for the "calendar" look, and a bit of **JavaScript** to handle the moving "red line" indicator.

Here is a complete, single-file solution you can save as an .html file and open in any browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Daily Class Schedule</title>
  <style>
    :root {
      --cell-height: 60px;
      --header-color: #2c3e50;
      --accent-color: #e74c3c;
      --bg-color: #f4f7f6;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: var(--bg-color);
      display: flex;
      justify-content: center;
      padding: 20px;
    }.calendar-container {
      width: 100%;
      max-width: 600px;
      background: white;
      border-radius: 8px;
      box-shadow: 0 4px 15px rgba(0,0,0,0.1);
      position: relative;
      overflow: hidden;
    }.calendar-header {
```

```
        background: var(--header-color);
        color: white;
        padding: 15px;
        text-align: center;
        font-size: 1.2rem;
        font-weight: bold;
    }.schedule-grid {
        position: relative;
        padding: 10px 0;
    }.time-slot {
        display: flex;
        height: var(--cell-height);
        border-bottom: 1px solid #eee;
        align-items: center;
    }.time-label {
        width: 120px;
        padding: 0 15px;
        font-size: 0.85rem;
        color: #666;
        font-weight: bold;
    }.subject-cell {
        flex-grow: 1;
        padding: 0 15px;
        height: 80%;
        display: flex;
        align-items: center;
        border-radius: 4px;
        margin-right: 10px;
        font-weight: 500;
    }

    /* Class specific colors */.class-active { background-color: #d1ecf1; color: #0c5460;
border-left: 4px solid #0c5460; }.free-slot { background-color: #f8f9fa; color: #adb5bd;
font-style: italic; }

    /* The Timeline Indicator */
    #timeline-indicator {
        position: absolute;
        left: 0;
        right: 0;
        height: 2px;
        background-color: var(--accent-color);
        z-index: 10;
```

```
        pointer-events: none;
        display: none; /* Hidden by default until JS finds current time */
      }

      #timeline-indicator::before {
        content: '';
        position: absolute;
        left: 0;
        top: -4px;
        width: 10px;
        height: 10px;
        background-color: var(--accent-color);
        border-radius: 50%;
      }
    </style>
</head>
<body>

<div class="calendar-container">
    <div class="calendar-header">Today's Schedule</div>

    <div class="schedule-grid" id="schedule-grid">
        <div id="timeline-indicator"></div>

        <div class="time-slot" data-start="09:00" data-end="09:30">
            <div class="time-label">09:00 – 09:30</div>
            <div class="subject-cell free-slot">Free / No Class</div>
        </div>

        <div class="time-slot" data-start="09:30" data-end="10:00">
            <div class="time-label">09:30 – 10:00</div>
            <div class="subject-cell free-slot">Free / No Class</div>
        </div>

        <div class="time-slot" data-start="10:00" data-end="10:30">
            <div class="time-label">10:00 – 10:30</div>
            <div class="subject-cell class-active">5A (NAFS M)</div>
        </div>

        <div class="time-slot" data-start="10:30" data-end="11:00">
            <div class="time-label">10:30 – 11:00</div>
            <div class="subject-cell free-slot">Free / No Class</div>
```

```
      </div>

      <div class="time-slot" data-start="11:00" data-end="11:30">
        <div class="time-label">11:00 – 11:30</div>
        <div class="subject-cell class-active">5B</div>
      </div>

      <div class="time-slot" data-start="11:30" data-end="12:00">
        <div class="time-label">11:30 – 12:00</div>
        <div class="subject-cell class-active">4A</div>
      </div>

      <div class="time-slot" data-start="12:00" data-end="12:25">
        <div class="time-label">12:00 – 12:25</div>
        <div class="subject-cell class-active">7A</div>
      </div>

      <div class="time-slot" data-start="12:25" data-end="12:50">
        <div class="time-label">12:25 – 12:50</div>
        <div class="subject-cell free-slot">Free / No Class</div>
      </div>
    </div>
  </div>

  <script>
    function updateTimeline() {
      const now = new Date();
      const currentTime = now.getHours() * 60 + now.getMinutes();

      const indicator = document.getElementById('timeline-indicator');
      const slots = document.querySelectorAll('.time-slot');
      let found = false;

      slots.forEach(slot => {
        const startStr = slot.getAttribute('data-start').split(':');
        const endStr = slot.getAttribute('data-end').split(':');

        const startMins = parseInt(startStr[0]) * 60 + parseInt(startStr[1]);
        const endMins = parseInt(endStr[0]) * 60 + parseInt(endStr[1]);

        if (currentTime >= startMins && currentTime <= endMins) {
          // Calculate position percentage within the slot
```

```
            const totalSlotMins = endMins - startMins;
            const minsPassed = currentTime - startMins;
            const percentDone = minsPassed / totalSlotMins;

            // Move indicator
            const slotTop = slot.offsetTop;
            const slotHeight = slot.offsetHeight;
            const indicatorPos = slotTop + (slotHeight * percentDone);

            indicator.style.display = 'block';
            indicator.style.top = indicatorPos + 'px';
            found = true;

            // Highlight current slot
            slot.style.backgroundColor = "rgba(231, 76, 60, 0.05)";
        } else {
            slot.style.backgroundColor = "transparent";
        }
    });

    if (!found) indicator.style.display = 'none';
}

// Run every minute
setInterval(updateTimeline, 60000);
// Run immediately on load
updateTimeline();
</script>

</body>
</html>
```

How it works

- **Data Attributes:** Each time-slot has data-start and data-end attributes. This makes it easy for the JavaScript to “read” the time logic without complex parsing.
- **The CSS “Red Line”:** The #timeline-indicator is an absolute-positioned div that sits on top of the schedule.

- **JavaScript Logic:** 1. It gets the current hour and minute.

  2. It converts those into “minutes from midnight” (e.g., 10:30 becomes $10 \times 60 + 30 = 630$). 3. It finds which slot that time falls into and calculates exactly how far down the line should be placed using the slot’s offsetTop.

A Note on Testing

Since this relies on the **current time** of your computer, you won’t see the red line if you open the page outside of the 9:00 AM – 12:50 PM window.

Would you like me to add a “Test Mode” button that lets you simulate different times of the day?

add 5 to 9 as period 1.add line indicator for current time

I have updated the code to include **Period 1 (08:30 – 09:00)** as requested. Since it is currently **8:56 AM**, you will see the red line indicator active in that first slot immediately upon opening the page.

I also adjusted the logic to ensure the line moves smoothly across the container based on the exact minute.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Class Schedule with Live Indicator</title>
  <style>
    :root {
      --cell-height: 70px;
      --indicator-color: #ff3b30;
      --border-color: #e0e0e0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica,
Arial, sans-serif;
      background-color: #f9f9fb;
      display: flex;
      justify-content: center;
```

```
    padding: 40px 20px;
}.calendar-card {
    width: 100%;
    max-width: 500px;
    background: white;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.05);
    position: relative;
    border: 1px solid var(--border-color);
}.header {
    padding: 20px;
    border-bottom: 1px solid var(--border-color);
    display: flex;
    justify-content: space-between;
    align-items: center;
}.header h2 { margin: 0; font-size: 1.2rem; color: #333; }.schedule-container {
    position: relative;
    padding: 10px 0;
}

/* The Moving Indicator Line */
#time-line {
    position: absolute;
    left: 0;
    right: 0;
    height: 2px;
    background: var(--indicator-color);
    z-index: 100;
    pointer-events: none;
    transition: top 0.3s ease;
    display: none;
}

#time-line::before {
    content: '';
    position: absolute;
    left: 0;
    top: -5px;
    width: 12px;
    height: 12px;
    background: var(--indicator-color);
    border-radius: 50%;
    box-shadow: 0 0 5px rgba(255, 59, 48, 0.4);
```

```
        }.row {
            display: flex;
            height: var(--cell-height);
            border-bottom: 1px dotted var(--border-color);
            align-items: center;
            position: relative;
        }.row:last-child { border-bottom: none; }.time-col {
            width: 110px;
            padding-left: 20px;
            font-size: 0.85rem;
            color: #888;
            font-weight: 600;
        }.subject-col {
            flex: 1;
            padding: 10px;
            margin: 5px 15px 5px 0;
            border-radius: 8px;
            display: flex;
            flex-direction: column;
            justify-content: center;
            font-size: 0.95rem;
            font-weight: 500;
        }

        /* Styling for different states */.is-class { background-color: #eef2ff; color: #3730a3;
border-left: 4px solid #4f46e5; }.is-free { background-color: #f3f4f6; color: #6b7280;
font-style: italic; border-left: 4px solid #d1d5db; }.period-num { font-size: 0.7rem; text-
transform: uppercase; opacity: 0.7; margin-bottom: 2px; }
    </style>
</head>
<body>

<div class="calendar-card">
    <div class="header">
        <h2>Daily Schedule</h2>
        <span id="current-clock" style="font-weight: bold; color: #4f46e5;"></span>
    </div>

    <div class="schedule-container" id="schedule">
        <div id="time-line"></div>

        <div class="row" data-start="08:30" data-end="09:00">
```

```
        <div class="time-col">08:30 – 09:00</div>
        <div class="subject-col is-class"><span class="period-num">Period 1</span>5 to
9</div>
      </div>

      <div class="row" data-start="09:00" data-end="09:30">
        <div class="time-col">09:00 – 09:30</div>
        <div class="subject-col is-free"><span class="period-num">Period 1
(Ext)</span>Free / No Class</div>
      </div>

      <div class="row" data-start="09:30" data-end="10:00">
        <div class="time-col">09:30 – 10:00</div>
        <div class="subject-col is-free"><span class="period-num">Period 2</span>Free /
No Class</div>
      </div>

      <div class="row" data-start="10:00" data-end="10:30">
        <div class="time-col">10:00 – 10:30</div>
        <div class="subject-col is-class"><span class="period-num">Period 3</span>5A
(NAFS M)</div>
      </div>

      <div class="row" data-start="10:30" data-end="11:00">
        <div class="time-col">10:30 – 11:00</div>
        <div class="subject-col is-free"><span class="period-num">Period 4</span>Free /
No Class</div>
      </div>

      <div class="row" data-start="11:00" data-end="11:30">
        <div class="time-col">11:00 – 11:30</div>
        <div class="subject-col is-class"><span class="period-num">Period
5</span>5B</div>
      </div>

      <div class="row" data-start="11:30" data-end="12:00">
        <div class="time-col">11:30 – 12:00</div>
        <div class="subject-col is-class"><span class="period-num">Period
6</span>4A</div>
      </div>

      <div class="row" data-start="12:00" data-end="12:25">
```

```
        <div class="time-col">12:00 – 12:25</div>
        <div class="subject-col is-class"><span class="period-num">Period
7</span>7A</div>
      </div>

      <div class="row" data-start="12:25" data-end="12:50">
        <div class="time-col">12:25 – 12:50</div>
        <div class="subject-col is-free"><span class="period-num">Period 8</span>Free /
No Class</div>
      </div>
    </div>
</div>

<script>
  function updateSchedule() {
    const now = new Date();
    const hours = now.getHours();
    const mins = now.getMinutes();
    const currentTimeInMins = hours * 60 + mins;

    // Update digital clock display
    document.getElementById('current-clock').innerText =
      now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

    const indicator = document.getElementById('time-line');
    const rows = document.querySelectorAll('.row');
    let indicatorSet = false;

    rows.forEach(row => {
      const start = row.getAttribute('data-start').split(':');
      const end = row.getAttribute('data-end').split(':');

      const startMins = parseInt(start[0]) * 60 + parseInt(start[1]);
      const endMins = parseInt(end[0]) * 60 + parseInt(end[1]);

      if (currentTimeInMins >= startMins && currentTimeInMins < endMins) {
        const totalDuration = endMins - startMins;
        const elapsed = currentTimeInMins - startMins;
        const percentOfRow = elapsed / totalDuration;

        const rowTop = row.offsetTop;
        const rowHeight = row.offsetHeight;
```

```
            indicator.style.display = 'block';
            indicator.style.top = (rowTop + (rowHeight * percentOfRow)) + 'px';
            row.style.backgroundColor = "rgba(79, 70, 229, 0.03)";
            indicatorSet = true;
        } else {
            row.style.backgroundColor = "transparent";
        }
    });

    if (!indicatorSet) {
        indicator.style.display = 'none';
    }
}

// Refresh every 30 seconds
setInterval(updateSchedule, 30000);
updateSchedule();
</script>

</body>
</html>
```

Key Changes

- **Period 1 Added:** I inserted a new slot from **08:30 to 09:00** titled “5 to 9”.
- **Precision Indicator:** The JavaScript now calculates the exact vertical position within each 30-minute block. If it’s 8:45 AM, the line will be exactly halfway through the first box.
- **Auto-Clock:** Added a digital clock in the header so you can verify the sync between your system time and the red line.

Would you like me to adjust any of the other time durations or add more classes to the list?

more clear visual 3d colors.free is blank

Here is the updated schedule with a much richer, **3D visual style**. I’ve added gradients, drop shadows, and inset shadows to give the active classes a distinct “raised card” look.

I have also completely cleared out the free periods so they appear as blank, recessed slots, exactly as you requested.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>3D Class Schedule with Live Timeline</title>
  <style>
    :root {
      --cell-height: 75px;
      --bg-color: #eef2f5;
      --indicator-color: #ff2a2a;
    }

    body {
      font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      display: flex;
      justify-content: center;
      padding: 40px 20px;
      margin: 0;
    }.calendar-card {
      width: 100%;
      max-width: 550px;
      background: #ffffff;
      border-radius: 16px;
      box-shadow:
        0 20px 40px rgba(0,0,0,0.1),
        0 1px 3px rgba(0,0,0,0.05),
        inset 0 2px 0 rgba(255,255,255,1);
      position: relative;
      border: 1px solid #d1d9e6;
      overflow: hidden;
    }.header {
      background: linear-gradient(180deg, #ffffff 0%, #f4f7f9 100%);
      padding: 20px 25px;
      border-bottom: 2px solid #e1e6eb;
      display: flex;
      justify-content: space-between;
      align-items: center;
      box-shadow: 0 4px 6px -4px rgba(0,0,0,0.05);
```

```
    }.header h2 {
        margin: 0;
        font-size: 1.3rem;
        color: #2c3e50;
        text-shadow: 1px 1px 0px rgba(255,255,255,1);
    }.clock {
        font-weight: 700;
        color: #3b82f6;
        background: #eef2f5;
        padding: 5px 12px;
        border-radius: 8px;
        box-shadow: inset 0 2px 4px rgba(0,0,0,0.05), 0 1px 0 rgba(255,255,255,1);
    }.schedule-container {
        position: relative;
        padding: 15px 0;
        background: #f8fafc;
    }

    /* 3D Moving Indicator Line */
    #time-line {
        position: absolute;
        left: 0;
        right: 0;
        height: 3px;
        background: linear-gradient(90deg, var(--indicator-color) 0%, #ff7b7b 50%,
transparent 100%);
        z-index: 100;
        pointer-events: none;
        transition: top 0.4s cubic-bezier(0.4, 0, 0.2, 1);
        display: none;
        box-shadow: 0 3px 5px rgba(255, 42, 42, 0.4);
    }

    #time-line::before {
        content: '';
        position: absolute;
        left: 0;
        top: -6px;
        width: 14px;
        height: 14px;
        background: var(--indicator-color);
        border-radius: 50%;
        border: 2px solid white;
```

```
    box-shadow: 0 0 10px rgba(255, 42, 42, 0.8), 0 2px 4px rgba(0,0,0,0.3);
}.row {
    display: flex;
    height: var(--cell-height);
    align-items: center;
    position: relative;
    margin-bottom: 5px;
}.time-col {
    width: 110px;
    padding-left: 20px;
    font-size: 0.85rem;
    color: #64748b;
    font-weight: 700;
    text-shadow: 1px 1px 0px white;
}.subject-col {
    flex: 1;
    height: 80%;
    margin: 0 20px 0 0;
    border-radius: 10px;
    display: flex;
    flex-direction: column;
    justify-content: center;
    padding: 0 15px;
    font-size: 1rem;
    font-weight: 600;
    transition: transform 0.2s ease;
}

/* 3D Active Class Styling */.is-class {
    background: linear-gradient(135deg, #3b82f6 0%, #2563eb 100%);
    color: white;
    box-shadow:
        0 6px 12px rgba(37, 99, 235, 0.25),
        inset 0 2px 0 rgba(255, 255, 255, 0.2),
        inset 0 -2px 0 rgba(0, 0, 0, 0.1);
    border: 1px solid #1d4ed8;
    text-shadow: 0 1px 2px rgba(0,0,0,0.2);
}

/* Hover effect to enhance 3D feel */.is-class:hover {
    transform: translateY(-2px);
    box-shadow:
        0 8px 15px rgba(37, 99, 235, 0.3),
```

```
            inset 0 2px 0 rgba(255, 255, 255, 0.3),
            inset 0 -2px 0 rgba(0, 0, 0, 0.1);
    }

    /* 3D Blank/Free Slot Styling (Recessed) */.is-free {
        background: transparent;
        box-shadow: inset 0 3px 6px rgba(0,0,0,0.04), inset 0 0 0 1px rgba(0,0,0,0.02);
        border: 1px dashed #cbd5e1;
    }.period-num {
        font-size: 0.75rem;
        text-transform: uppercase;
        opacity: 0.85;
        margin-bottom: 3px;
        letter-spacing: 0.5px;
    }

  </style>
</head>
<body>

<div class="calendar-card">
  <div class="header">
    <h2>Daily Schedule</h2>
    <span id="current-clock" class="clock"></span>
  </div>

  <div class="schedule-container" id="schedule">
    <div id="time-line"></div>

    <div class="row" data-start="08:30" data-end="09:00">
      <div class="time-col">08:30 – 09:00</div>
      <div class="subject-col is-class">
        <span class="period-num">Period 1</span>
        5 to 9
      </div>
    </div>

    <div class="row" data-start="09:00" data-end="09:30">
      <div class="time-col">09:00 – 09:30</div>
      <div class="subject-col is-free"></div>
    </div>
```

```
<div class="row" data-start="09:30" data-end="10:00">
  <div class="time-col">09:30 – 10:00</div>
  <div class="subject-col is-free"></div>
</div>

<div class="row" data-start="10:00" data-end="10:30">
  <div class="time-col">10:00 – 10:30</div>
  <div class="subject-col is-class">
    <span class="period-num">Period 3</span>
    5A (NAFS M)
  </div>
</div>

<div class="row" data-start="10:30" data-end="11:00">
  <div class="time-col">10:30 – 11:00</div>
   <div class="subject-col is-free"></div>
</div>

<div class="row" data-start="11:00" data-end="11:30">
  <div class="time-col">11:00 – 11:30</div>
  <div class="subject-col is-class">
    <span class="period-num">Period 5</span>
    5B
  </div>
</div>

<div class="row" data-start="11:30" data-end="12:00">
  <div class="time-col">11:30 – 12:00</div>
  <div class="subject-col is-class">
    <span class="period-num">Period 6</span>
    4A
  </div>
</div>

<div class="row" data-start="12:00" data-end="12:25">
  <div class="time-col">12:00 – 12:25</div>
  <div class="subject-col is-class">
    <span class="period-num">Period 7</span>
    7A
  </div>
</div>
```

```
      <div class="row" data-start="12:25" data-end="12:50">
        <div class="time-col">12:25 – 12:50</div>
         <div class="subject-col is-free"></div>
      </div>
    </div>
</div>

<script>
    function updateSchedule() {
      const now = new Date();
      const hours = now.getHours();
      const mins = now.getMinutes();
      const currentTimeInMins = hours * 60 + mins;

      // Update digital clock display
      document.getElementById('current-clock').innerText =
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

      const indicator = document.getElementById('time-line');
      const rows = document.querySelectorAll('.row');
      let indicatorSet = false;

      rows.forEach(row => {
        const start = row.getAttribute('data-start').split(':');
        const end = row.getAttribute('data-end').split(':');

        const startMins = parseInt(start[0]) * 60 + parseInt(start[1]);
        const endMins = parseInt(end[0]) * 60 + parseInt(end[1]);

        if (currentTimeInMins >= startMins && currentTimeInMins < endMins) {
          const totalDuration = endMins - startMins;
          const elapsed = currentTimeInMins - startMins;
          const percentOfRow = elapsed / totalDuration;

          const rowTop = row.offsetTop;
          const rowHeight = row.offsetHeight;

          indicator.style.display = 'block';
          // Adjusting slightly so the line sits right inside the box perfectly
          indicator.style.top = (rowTop + (rowHeight * percentOfRow)) + 'px';

          indicatorSet = true;
```

```
        }
      });

      if (!indicatorSet) {
        indicator.style.display = 'none';
      }
    }

    // Refresh every 30 seconds
    setInterval(updateSchedule, 30000);
    updateSchedule();
</script>

</body>
</html>
```

Visual Updates Made

- **3D "Raised" Classes:** Active class blocks now use gradient backgrounds, top-edge highlights (inset white shadow), and drop shadows to look like physical buttons or cards.
- **"Blank" Free Slots:** The text has been entirely removed from the free slots. They now feature a faint inset shadow and a dashed border to look like an empty, recessed "groove" waiting to be filled.
- **Glowing Indicator Line:** The red timeline indicator now has a soft drop shadow and a glowing dot on the left side, making it stand out perfectly against the 3D elements.
- **Modern Header:** Added a sleek, embedded digital clock with a 3D inset background to match the aesthetic.

Would you like me to show you how to customize the colors of the class blocks so each class (like 5A vs 4A) has its own unique 3D color?

> can this page read from json file: if so :json file with the whole scadual : list of dayes :open to curent day:btn to current day

Yes, we can absolutely make the page read from a JSON structure.

To make this easy to run as a single file without needing to set up a local web server (which is normally required to fetch external .json files due to browser

security rules), I have embedded the JSON data directly into the JavaScript as a constant variable.

I have fully transcribed the schedule from your image (Sunday through Thursday). The app now features **day tabs**, automatically opens to the **current day**, and includes a **"Go to Today"** button.

Complete Dynamic Schedule Code

Save the following code as an .html file and open it in your browser:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic 3D Class Schedule</title>
  <style>
    :root {
      --cell-height: 70px;
      --bg-color: #eef2f5;
      --indicator-color: #ff2a2a;
      --primary-blue: #3b82f6;
      --primary-dark: #2563eb;
    }

    body {
      font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      display: flex;
      justify-content: center;
      padding: 40px 20px;
      margin: 0;
    }.calendar-card {
      width: 100%;
      max-width: 600px;
      background: #ffffff;
      border-radius: 16px;
      box-shadow: 0 20px 40px rgba(0,0,0,0.1), 0 1px 3px rgba(0,0,0,0.05);
      border: 1px solid #d1d9e6;
      overflow: hidden;
      display: flex;
      flex-direction: column;
```

```
        }.header {
            background: linear-gradient(180deg, #ffffff 0%, #f4f7f9 100%);
            padding: 20px 25px;
            display: flex;
            justify-content: space-between;
            align-items: center;
            border-bottom: 1px solid #e1e6eb;
        }.header-title {
            display: flex;
            flex-direction: column;
        }.header h2 { margin: 0; font-size: 1.4rem; color: #2c3e50; }.current-day-label { font-
size: 0.85rem; color: #64748b; font-weight: 600; margin-top: 4px; }.clock {
            font-weight: 700;
            color: var(--primary-blue);
            background: #eef2f5;
            padding: 8px 15px;
            border-radius: 8px;
            box-shadow: inset 0 2px 4px rgba(0,0,0,0.05), 0 1px 0 rgba(255,255,255,1);
        }

        /* Day Navigation Tabs */.day-nav {
            display: flex;
            background: #f8fafc;
            padding: 10px 15px;
            gap: 8px;
            border-bottom: 2px solid #e1e6eb;
            align-items: center;
            overflow-x: auto;
        }.day-btn {
            background: transparent;
            border: 1px solid transparent;
            padding: 8px 12px;
            border-radius: 8px;
            font-weight: 600;
            color: #64748b;
            cursor: pointer;
            transition: all 0.2s;
            flex-shrink: 0;
        }.day-btn:hover { background: #e2e8f0; color: #334155; }.day-btn.active {
            background: white;
            color: var(--primary-dark);
            border: 1px solid #cbd5e1;
            box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05), 0 2px 4px -1px rgba(0,0,0,0.03);
```

```
    }.today-btn {
        margin-left: auto;
        background: linear-gradient(135deg, #10b981 0%, #059669 100%);
        color: white;
        border: none;
        padding: 8px 15px;
        border-radius: 8px;
        font-weight: 700;
        cursor: pointer;
        box-shadow: 0 4px 6px rgba(16, 185, 129, 0.2), inset 0 2px 0 rgba(255,255,255,0.2);
        font-size: 0.85rem;
    }.today-btn:hover { transform: translateY(-1px); box-shadow: 0 6px 8px rgba(16,
185, 129, 0.3); }

    /* Schedule Grid */.schedule-container {
        position: relative;
        padding: 15px 0;
        background: #f8fafc;
        min-height: 400px;
    }

    #time-line {
        position: absolute;
        left: 0;
        right: 0;
        height: 3px;
        background: linear-gradient(90deg, var(--indicator-color) 0%, #ff7b7b 50%,
transparent 100%);
        z-index: 100;
        pointer-events: none;
        transition: top 0.4s cubic-bezier(0.4, 0, 0.2, 1);
        display: none;
    }

    #time-line::before {
        content: '';
        position: absolute;
        left: 0;
        top: -6px;
        width: 14px;
        height: 14px;
        background: var(--indicator-color);
```

```
            border-radius: 50%;
            border: 2px solid white;
            box-shadow: 0 0 10px rgba(255, 42, 42, 0.6);
        }.row {
            display: flex;
            height: var(--cell-height);
            align-items: center;
            position: relative;
            margin-bottom: 8px;
        }.time-col {
            width: 120px;
            padding-left: 20px;
            font-size: 0.8rem;
            color: #64748b;
            font-weight: 700;
            display: flex;
            flex-direction: column;
        }.time-col.period-number { font-size: 0.7rem; color: #94a3b8; text-transform:
uppercase; }.subject-col {
            flex: 1;
            height: 85%;
            margin: 0 20px 0 0;
            border-radius: 10px;
            display: flex;
            flex-direction: column;
            justify-content: center;
            padding: 0 15px;
            font-size: 1.05rem;
            font-weight: 700;
        }

        /* 3D Styling */.is-class {
            background: linear-gradient(135deg, var(--primary-blue) 0%, var(--primary-dark)
100%);
            color: white;
            box-shadow: 0 6px 12px rgba(37, 99, 235, 0.25), inset 0 2px 0 rgba(255, 255, 255,
0.2), inset 0 -2px 0 rgba(0, 0, 0, 0.1);
            border: 1px solid #1d4ed8;
            text-shadow: 0 1px 2px rgba(0,0,0,0.2);
        }.is-free {
            background: transparent;
            box-shadow: inset 0 3px 6px rgba(0,0,0,0.04);
            border: 1px dashed #cbd5e1;
```

```
    }
  </style>
</head>
<body>

<div class="calendar-card">
  <div class="header">
    <div class="header-title">
      <h2>Weekly Schedule</h2>
      <span class="current-day-label" id="display-day-name">Loading...</span>
    </div>
    <span id="current-clock" class="clock">00:00</span>
  </div>

  <div class="day-nav" id="day-nav">
    </div>

  <div class="schedule-container">
    <div id="time-line"></div>
    <div id="schedule-body">
      </div>
  </div>
</div>

<script>
  // --- 1. JSON DATA STRUCTURE ---
  // Transcribed exactly from your uploaded image
  const scheduleData = {
    0: [ // Sunday
      { period: 1, start: "09:00", end: "09:30", subject: "7A" },
      { period: 2, start: "09:30", end: "10:00", subject: "" },
      { period: 3, start: "10:00", end: "10:30", subject: "4A" },
      { period: 4, start: "10:30", end: "11:00", subject: "" },
      { period: 5, start: "11:00", end: "11:30", subject: "5B" },
      { period: 6, start: "11:30", end: "12:00", subject: "5A" },
      { period: 7, start: "12:00", end: "12:25", subject: "" },
      { period: 8, start: "12:25", end: "12:50", subject: "" }
    ],
    1: [ // Monday
      { period: 1, start: "09:00", end: "09:30", subject: "7A (NAFS M)" },
      { period: 2, start: "09:30", end: "10:00", subject: "" },
      { period: 3, start: "10:00", end: "10:30", subject: "" },
```

```
            { period: 4, start: "10:30", end: "11:00", subject: "5B" },
            { period: 5, start: "11:00", end: "11:30", subject: "" },
            { period: 6, start: "11:30", end: "12:00", subject: "4A" },
            { period: 7, start: "12:00", end: "12:25", subject: "7A" },
            { period: 8, start: "12:25", end: "12:50", subject: "5A" }
        ],
        2: [ // Tuesday
            { period: 1, start: "09:00", end: "09:30", subject: "" },
            { period: 2, start: "09:30", end: "10:00", subject: "" },
            { period: 3, start: "10:00", end: "10:30", subject: "5A (NAFS M)" },
            { period: 4, start: "10:30", end: "11:00", subject: "" },
            { period: 5, start: "11:00", end: "11:30", subject: "5B" },
            { period: 6, start: "11:30", end: "12:00", subject: "4A" },
            { period: 7, start: "12:00", end: "12:25", subject: "7A" },
            { period: 8, start: "12:25", end: "12:50", subject: "" }
        ],
        3: [ // Wednesday
            { period: 1, start: "09:00", end: "09:30", subject: "" },
            { period: 2, start: "09:30", end: "10:00", subject: "" },
            { period: 3, start: "10:00", end: "10:30", subject: "" },
            { period: 4, start: "10:30", end: "11:00", subject: "5B" },
            { period: 5, start: "11:00", end: "11:30", subject: "7A" },
            { period: 6, start: "11:30", end: "12:00", subject: "" },
            { period: 7, start: "12:00", end: "12:25", subject: "5A" },
            { period: 8, start: "12:25", end: "12:50", subject: "" }
        ],
        4: [ // Thursday
            { period: 1, start: "09:00", end: "09:30", subject: "" },
            { period: 2, start: "09:30", end: "10:00", subject: "" },
            { period: 3, start: "10:00", end: "10:30", subject: "4A (NAFS M)" },
            { period: 4, start: "10:30", end: "11:00", subject: "4A" },
            { period: 5, start: "11:00", end: "11:30", subject: "5B (NAFS M)" },
            { period: 6, start: "11:30", end: "12:00", subject: "" },
            { period: 7, start: "12:00", end: "12:25", subject: "5A" },
            { period: 8, start: "12:25", end: "12:50", subject: "" }
        ]
    };

    const dayNames = { 0: "Sunday", 1: "Monday", 2: "Tuesday", 3: "Wednesday", 4:
"Thursday" };
    let viewingDayIndex = 0; // The day currently displayed on screen

    // --- 2. INITIALIZATION ---
```

```
    function init() {
        buildDayTabs();
        goToToday(); // Automatically opens to the current day

        // Update the clock and timeline every 30 seconds
        setInterval(updateTimeAndIndicator, 30000);
    }

    // --- 3. UI RENDERING ---
    function buildDayTabs() {
        const nav = document.getElementById('day-nav');
        nav.innerHTML = ''; // Clear

        for (let i = 0; i <= 4; i++) {
            const btn = document.createElement('button');
            btn.className = `day-btn`;
            btn.innerText = dayNames[i].substring(0, 3); // "Sun", "Mon", etc.
            btn.onclick = () => loadScheduleForDay(i);
            nav.appendChild(btn);
        }

        const todayBtn = document.createElement('button');
        todayBtn.className = 'today-btn';
        todayBtn.innerText = 'Go to Today';
        todayBtn.onclick = goToToday;
        nav.appendChild(todayBtn);
    }

    function loadScheduleForDay(dayIndex) {
        viewingDayIndex = dayIndex;

        // Update header text
        document.getElementById('display-day-name').innerText = dayNames[dayIndex] +
"'s Schedule";

        // Update active tab styles
        document.querySelectorAll('.day-btn').forEach((btn, idx) => {
            btn.classList.toggle('active', idx === dayIndex);
        });

        // Render the rows
        const container = document.getElementById('schedule-body');
```

```
    container.innerHTML = '';

    const dayData = scheduleData[dayIndex];

    dayData.forEach(slot => {
        const isFree = slot.subject === "";
        const subjectClass = isFree? "is-free" : "is-class";
        const subjectText = isFree? "" : slot.subject;

        const rowHTML = `
            <div class="row" data-start="${slot.start}" data-end="${slot.end}">
                <div class="time-col">
                    <span class="period-number">Period ${slot.period}</span>
                    <span>${slot.start} – ${slot.end}</span>
                </div>
                <div class="subject-col ${subjectClass}">${subjectText}</div>
            </div>
        `;
        container.insertAdjacentHTML('beforeend', rowHTML);
    });

    // After rendering rows, calculate where the timeline should be
    updateTimeAndIndicator();
}

// --- 4. LOGIC & TIMELINE ---
function goToToday() {
    let currentRealDay = new Date().getDay();
    // JS Date: 0=Sun, 1=Mon... 5=Fri, 6=Sat

    // If it's Friday or Saturday, default to Sunday
    if (currentRealDay > 4) currentRealDay = 0;

    loadScheduleForDay(currentRealDay);
}

function updateTimeAndIndicator() {
    const now = new Date();
    const currentRealDay = now.getDay();
    const hours = now.getHours();
    const mins = now.getMinutes();
    const currentTimeInMins = hours * 60 + mins;
```

```
    // Update digital clock
    document.getElementById('current-clock').innerText =
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

    const indicator = document.getElementById('time-line');
    const rows = document.querySelectorAll('.row');
    let indicatorSet = false;

    // ONLY show the red line if the user is looking at TODAY'S schedule
    if (viewingDayIndex === currentRealDay) {
        rows.forEach(row => {
            const start = row.getAttribute('data-start').split(':');
            const end = row.getAttribute('data-end').split(':');

            const startMins = parseInt(start[0]) * 60 + parseInt(start[1]);
            const endMins = parseInt(end[0]) * 60 + parseInt(end[1]);

            if (currentTimeInMins >= startMins && currentTimeInMins < endMins) {
                const totalDuration = endMins - startMins;
                const elapsed = currentTimeInMins - startMins;
                const percentOfRow = elapsed / totalDuration;

                const rowTop = row.offsetTop;
                const rowHeight = row.offsetHeight;

                indicator.style.display = 'block';
                // Move indicator to current progression inside the row
                indicator.style.top = (rowTop + (rowHeight * percentOfRow)) + 'px';

                indicatorSet = true;
            }
        });
    }

    if (!indicatorSet) {
        indicator.style.display = 'none';
    }
}

// Start everything
init();
```

```
</script>

</body>
</html>
```

How this specific logic works:

1. **The JSON Dictionary:** const scheduleData = {... } acts as your JSON data source. Each day (0 to 4) is an array of objects representing the periods.
2. **Current Day Check:** new Date().getDay() checks your system clock. It automatically figures out if today is Tuesday, translates that to 2, and runs the loadScheduleForDay(2) function on page load. *(Note: If you open it on a Friday or Saturday, I programmed it to safely default back to Sunday).*
3. **Smart Timeline:** The red line indicator will **only** show up if the day tab you are currently looking at matches your actual real-world day. If you are viewing Monday's schedule on a Tuesday, the timeline correctly hides itself.

Would you like me to map the specific yellow and light-blue colors from your image to their respective subjects (e.g., all 7A classes turn yellow, 4A turn light blue)?

json and page code

To read from an external JSON file, we need to split this into two separate files: schedule.json and index.html.

Here is the exact code for both files.

1. The JSON File (schedule.json)

Create a new file named schedule.json, copy this text into it, and save it in the exact same folder as your HTML file.

```
{
  "0": [
    { "period": 1, "start": "09:00", "end": "09:30", "subject": "7A" },
    { "period": 2, "start": "09:30", "end": "10:00", "subject": "" },
    { "period": 3, "start": "10:00", "end": "10:30", "subject": "4A" },
    { "period": 4, "start": "10:30", "end": "11:00", "subject": "" },
    { "period": 5, "start": "11:00", "end": "11:30", "subject": "5B" },
```

```
  { "period": 6, "start": "11:30", "end": "12:00", "subject": "5A" },
  { "period": 7, "start": "12:00", "end": "12:25", "subject": "" },
  { "period": 8, "start": "12:25", "end": "12:50", "subject": "" }
],
"1": [
  { "period": 1, "start": "09:00", "end": "09:30", "subject": "7A (NAFS M)" },
  { "period": 2, "start": "09:30", "end": "10:00", "subject": "" },
  { "period": 3, "start": "10:00", "end": "10:30", "subject": "" },
  { "period": 4, "start": "10:30", "end": "11:00", "subject": "5B" },
  { "period": 5, "start": "11:00", "end": "11:30", "subject": "" },
  { "period": 6, "start": "11:30", "end": "12:00", "subject": "4A" },
  { "period": 7, "start": "12:00", "end": "12:25", "subject": "7A" },
  { "period": 8, "start": "12:25", "end": "12:50", "subject": "5A" }
],
"2": [
  { "period": 1, "start": "09:00", "end": "09:30", "subject": "" },
  { "period": 2, "start": "09:30", "end": "10:00", "subject": "" },
  { "period": 3, "start": "10:00", "end": "10:30", "subject": "5A (NAFS M)" },
  { "period": 4, "start": "10:30", "end": "11:00", "subject": "" },
  { "period": 5, "start": "11:00", "end": "11:30", "subject": "5B" },
  { "period": 6, "start": "11:30", "end": "12:00", "subject": "4A" },
  { "period": 7, "start": "12:00", "end": "12:25", "subject": "7A" },
  { "period": 8, "start": "12:25", "end": "12:50", "subject": "" }
],
"3": [
  { "period": 1, "start": "09:00", "end": "09:30", "subject": "" },
  { "period": 2, "start": "09:30", "end": "10:00", "subject": "" },
  { "period": 3, "start": "10:00", "end": "10:30", "subject": "" },
  { "period": 4, "start": "10:30", "end": "11:00", "subject": "5B" },
  { "period": 5, "start": "11:00", "end": "11:30", "subject": "7A" },
  { "period": 6, "start": "11:30", "end": "12:00", "subject": "" },
  { "period": 7, "start": "12:00", "end": "12:25", "subject": "5A" },
  { "period": 8, "start": "12:25", "end": "12:50", "subject": "" }
],
"4": [
  { "period": 1, "start": "09:00", "end": "09:30", "subject": "" },
  { "period": 2, "start": "09:30", "end": "10:00", "subject": "" },
  { "period": 3, "start": "10:00", "end": "10:30", "subject": "4A (NAFS M)" },
  { "period": 4, "start": "10:30", "end": "11:00", "subject": "4A" },
  { "period": 5, "start": "11:00", "end": "11:30", "subject": "5B (NAFS M)" },
  { "period": 6, "start": "11:30", "end": "12:00", "subject": "" },
  { "period": 7, "start": "12:00", "end": "12:25", "subject": "5A" },
  { "period": 8, "start": "12:25", "end": "12:50", "subject": "" }
```

```
  ]
}
```

---

2. The HTML Page (index.html)

Save this as index.html in the exact same folder as the schedule.json file. It now uses the fetch() API to load the schedule dynamically.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic 3D Class Schedule</title>
  <style>
    :root {
      --cell-height: 70px;
      --bg-color: #eef2f5;
      --indicator-color: #ff2a2a;
      --primary-blue: #3b82f6;
      --primary-dark: #2563eb;
    }

    body {
      font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      display: flex;
      justify-content: center;
      padding: 40px 20px;
      margin: 0;
    }.calendar-card {
      width: 100%;
      max-width: 600px;
      background: #ffffff;
      border-radius: 16px;
      box-shadow: 0 20px 40px rgba(0,0,0,0.1), 0 1px 3px rgba(0,0,0,0.05);
      border: 1px solid #d1d9e6;
      overflow: hidden;
      display: flex;
      flex-direction: column;
    }.header {
      background: linear-gradient(180deg, #ffffff 0%, #f4f7f9 100%);
```

```
        padding: 20px 25px;
        display: flex;
        justify-content: space-between;
        align-items: center;
        border-bottom: 1px solid #e1e6eb;
    }.header-title {
        display: flex;
        flex-direction: column;
    }.header h2 { margin: 0; font-size: 1.4rem; color: #2c3e50; }.current-day-label { font-
size: 0.85rem; color: #64748b; font-weight: 600; margin-top: 4px; }.clock {
        font-weight: 700;
        color: var(--primary-blue);
        background: #eef2f5;
        padding: 8px 15px;
        border-radius: 8px;
        box-shadow: inset 0 2px 4px rgba(0,0,0,0.05), 0 1px 0 rgba(255,255,255,1);
    }

    /* Day Navigation Tabs */.day-nav {
        display: flex;
        background: #f8fafc;
        padding: 10px 15px;
        gap: 8px;
        border-bottom: 2px solid #e1e6eb;
        align-items: center;
        overflow-x: auto;
    }.day-btn {
        background: transparent;
        border: 1px solid transparent;
        padding: 8px 12px;
        border-radius: 8px;
        font-weight: 600;
        color: #64748b;
        cursor: pointer;
        transition: all 0.2s;
        flex-shrink: 0;
    }.day-btn:hover { background: #e2e8f0; color: #334155; }.day-btn.active {
        background: white;
        color: var(--primary-dark);
        border: 1px solid #cbd5e1;
        box-shadow: 0 4px 6px -1px rgba(0,0,0,0.05), 0 2px 4px -1px rgba(0,0,0,0.03);
    }.today-btn {
        margin-left: auto;
```

```
            background: linear-gradient(135deg, #10b981 0%, #059669 100%);
            color: white;
            border: none;
            padding: 8px 15px;
            border-radius: 8px;
            font-weight: 700;
            cursor: pointer;
            box-shadow: 0 4px 6px rgba(16, 185, 129, 0.2), inset 0 2px 0 rgba(255,255,255,0.2);
            font-size: 0.85rem;
        }.today-btn:hover { transform: translateY(-1px); box-shadow: 0 6px 8px rgba(16, 185, 129, 0.3); }

        /* Schedule Grid */.schedule-container {
            position: relative;
            padding: 15px 0;
            background: #f8fafc;
            min-height: 400px;
        }

        #time-line {
            position: absolute;
            left: 0;
            right: 0;
            height: 3px;
            background: linear-gradient(90deg, var(--indicator-color) 0%, #ff7b7b 50%, transparent 100%);
            z-index: 100;
            pointer-events: none;
            transition: top 0.4s cubic-bezier(0.4, 0, 0.2, 1);
            display: none;
        }

        #time-line::before {
            content: '';
            position: absolute;
            left: 0;
            top: -6px;
            width: 14px;
            height: 14px;
            background: var(--indicator-color);
            border-radius: 50%;
            border: 2px solid white;
```

```
        box-shadow: 0 0 10px rgba(255, 42, 42, 0.6);
    }.row {
        display: flex;
        height: var(--cell-height);
        align-items: center;
        position: relative;
        margin-bottom: 8px;
    }.time-col {
        width: 120px;
        padding-left: 20px;
        font-size: 0.8rem;
        color: #64748b;
        font-weight: 700;
        display: flex;
        flex-direction: column;
    }.time-col.period-number { font-size: 0.7rem; color: #94a3b8; text-transform:
uppercase; }.subject-col {
        flex: 1;
        height: 85%;
        margin: 0 20px 0 0;
        border-radius: 10px;
        display: flex;
        flex-direction: column;
        justify-content: center;
        padding: 0 15px;
        font-size: 1.05rem;
        font-weight: 700;
    }

    /* 3D Styling */.is-class {
        background: linear-gradient(135deg, var(--primary-blue) 0%, var(--primary-dark)
100%);
        color: white;
        box-shadow: 0 6px 12px rgba(37, 99, 235, 0.25), inset 0 2px 0 rgba(255, 255, 255,
0.2), inset 0 -2px 0 rgba(0, 0, 0, 0.1);
        border: 1px solid #1d4ed8;
        text-shadow: 0 1px 2px rgba(0,0,0,0.2);
    }.is-free {
        background: transparent;
        box-shadow: inset 0 3px 6px rgba(0,0,0,0.04);
        border: 1px dashed #cbd5e1;
    }
  </style>
```

```
</head>
<body>

<div class="calendar-card">
  <div class="header">
    <div class="header-title">
      <h2>Weekly Schedule</h2>
      <span class="current-day-label" id="display-day-name">Loading...</span>
    </div>
    <span id="current-clock" class="clock">00:00</span>
  </div>

  <div class="day-nav" id="day-nav">
    </div>

  <div class="schedule-container">
    <div id="time-line"></div>
    <div id="schedule-body">
      </div>
  </div>
</div>

<script>
  let scheduleData = {};
  const dayNames = { 0: "Sunday", 1: "Monday", 2: "Tuesday", 3: "Wednesday", 4:
"Thursday" };
  let viewingDayIndex = 0;

  // --- 1. FETCH JSON DATA ---
  async function init() {
    try {
      // Fetching data from the external schedule.json file
      const response = await fetch('schedule.json');
      scheduleData = await response.json();

      buildDayTabs();
      goToToday();
      setInterval(updateTimeAndIndicator, 30000);
    } catch (error) {
      console.error("Failed to load schedule.json:", error);
      document.getElementById('display-day-name').innerText = "Error loading
schedule data.";
```

```
        document.getElementById('schedule-body').innerHTML = "<p style='padding: 20px; color: red;'>Could not load schedule.json. Make sure you are running a local web server.</p>";
      }
    }

    // --- 2. UI RENDERING ---
    function buildDayTabs() {
      const nav = document.getElementById('day-nav');
      nav.innerHTML = '';

      for (let i = 0; i <= 4; i++) {
        const btn = document.createElement('button');
        btn.className = `day-btn`;
        btn.innerText = dayNames[i].substring(0, 3);
        btn.onclick = () => loadScheduleForDay(i);
        nav.appendChild(btn);
      }

      const todayBtn = document.createElement('button');
      todayBtn.className = 'today-btn';
      todayBtn.innerText = 'Go to Today';
      todayBtn.onclick = goToToday;
      nav.appendChild(todayBtn);
    }

    function loadScheduleForDay(dayIndex) {
      viewingDayIndex = dayIndex;

      document.getElementById('display-day-name').innerText = dayNames[dayIndex] + "'s Schedule";

      document.querySelectorAll('.day-btn').forEach((btn, idx) => {
        btn.classList.toggle('active', idx === dayIndex);
      });

      const container = document.getElementById('schedule-body');
      container.innerHTML = '';

      const dayData = scheduleData[dayIndex];

      dayData.forEach(slot => {
```

```
        const isFree = slot.subject === "";
        const subjectClass = isFree? "is-free" : "is-class";
        const subjectText = isFree? "" : slot.subject;

        const rowHTML = `
          <div class="row" data-start="${slot.start}" data-end="${slot.end}">
            <div class="time-col">
              <span class="period-number">Period ${slot.period}</span>
              <span>${slot.start} – ${slot.end}</span>
            </div>
            <div class="subject-col ${subjectClass}">${subjectText}</div>
          </div>
        `;
        container.insertAdjacentHTML('beforeend', rowHTML);
    });

    updateTimeAndIndicator();
}

// --- 3. LOGIC & TIMELINE ---
function goToToday() {
    let currentRealDay = new Date().getDay();
    if (currentRealDay > 4) currentRealDay = 0;
    loadScheduleForDay(currentRealDay);
}

function updateTimeAndIndicator() {
    const now = new Date();
    const currentRealDay = now.getDay();
    const hours = now.getHours();
    const mins = now.getMinutes();
    const currentTimeInMins = hours * 60 + mins;

    document.getElementById('current-clock').innerText =
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

    const indicator = document.getElementById('time-line');
    const rows = document.querySelectorAll('.row');
    let indicatorSet = false;

    if (viewingDayIndex === currentRealDay) {
        rows.forEach(row => {
```

```
            const start = row.getAttribute('data-start').split(':');
            const end = row.getAttribute('data-end').split(':');

            const startMins = parseInt(start[0]) * 60 + parseInt(start[1]);
            const endMins = parseInt(end[0]) * 60 + parseInt(end[1]);

            if (currentTimeInMins >= startMins && currentTimeInMins < endMins) {
              const totalDuration = endMins - startMins;
              const elapsed = currentTimeInMins - startMins;
              const percentOfRow = elapsed / totalDuration;

              const rowTop = row.offsetTop;
              const rowHeight = row.offsetHeight;

              indicator.style.display = 'block';
              indicator.style.top = (rowTop + (rowHeight * percentOfRow)) + 'px';

              indicatorSet = true;
            }
          });
        }

        if (!indicatorSet) {
          indicator.style.display = 'none';
        }
      }

      // Start everything
      init();

    </script>

    </body>
    </html>
```

⚠️ Important Note on Fetching JSON Files

Web browsers have strict security policies (CORS). If you simply double-click index.html to open it as a file (your address bar will say file:///C:/...), the browser **will block** the script from reading schedule.json, and the schedule will not load.

To make it work, you must run it through a local web server. The easiest way is:

1. If you use **VS Code**, install the “Live Server” extension, right-click index.html, and select “Open with Live Server”.
2. If you have **Python** installed, open your terminal in that folder and run python -m http.server 8000, then go to http://localhost:8000 in your browser.

Would you like me to map the specific colors from your image (yellow for 7A, light blue for 4A) using the new JSON setup?

all scadual

If by “all schedule” you mean you want to see the **entire week at once in a full grid**—exactly like the image you uploaded—I have built that for you here!

This version recreates the full table layout. It automatically colors **7A yellow** and **4A light blue**, exactly as shown in your picture. It also features a “Live Highlighter” that puts a glowing red border around your **current active class** based on the current day and time.

Here is the complete, single-file code. You can save this as an .html file and open it in your browser immediately (no local server required for this one, as I embedded the data to make it easy for you).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Full Weekly Schedule</title>
  <style>
    :root {
      --header-bg: #4a4a4a;
      --day-bg: #5a5a5a;
      --day-text: #fce4ce;
      --border-color: #222;
      --7a-color: #ffff00;
      --4a-color: #dbeafe;
      --other-color: #ffffff;
      --live-glow: #ff2a2a;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
    background-color: #f0f2f5;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

h2 { margin: 0; color: #333; }.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

table {
    width: 100%;
    max-width: 1000px;
    border-collapse: collapse;
    background: white;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    border: 2px solid var(--border-color);
}

th, td {
    border: 1px solid var(--border-color);
    text-align: center;
    padding: 12px 5px;
    width: 11%;
}

/* Headers */
```

```
        th {
            background-color: var(--header-bg);
            color: white;
            font-size: 0.9rem;
            line-height: 1.4;
        }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }

        /* Day Column */.day-col {
            background-color: var(--day-bg);
            color: var(--day-text);
            font-size: 1.1rem;
            font-weight: bold;
            text-align: left;
            padding-left: 15px;
            width: 12%;
        }

        /* Subjects */.subject-cell {
            font-size: 1.2rem;
            font-weight: 900;
            color: #111;
            position: relative;
            transition: all 0.3s;
        }.nafs-text {
            display: block;
            font-size: 0.7rem;
            font-weight: normal;
            color: #444;
            margin-top: 5px;
        }

        /* Specific Colors from Image */.sub-7A { background-color: var(--7a-color); }.sub-4A
{ background-color: var(--4a-color); }.sub-other { background-color: var(--other-
color); }.sub-free { background-color: white; }

        /* Live Indicator - Highlights the current active class */.is-current-class {
            box-shadow: inset 0 0 0 4px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.6);
            z-index: 10;
            transform: scale(1.05);
            border-radius: 4px;
        }.is-current-class::after {
```

```
            content: 'LIVE';
            position: absolute;
            top: -10px;
            right: -10px;
            background: var(--live-glow);
            color: white;
            font-size: 0.6rem;
            padding: 2px 6px;
            border-radius: 10px;
            font-weight: bold;
            box-shadow: 0 2px 4px rgba(0,0,0,0.3);
        }.is-current-day {
            border-left: 5px solid var(--live-glow);
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2>Full Weekly View</h2>
    <div id="clock" class="clock">00:00</div>
</div>

<table id="schedule-table">
    <thead>
        <tr>
            <th>Day</th>
            <th><span class="period-num">1</span><span class="period-time">(9:00 -
9:30)</span></th>
            <th><span class="period-num">2</span><span class="period-time">(9:30 -
10:00)</span></th>
            <th><span class="period-num">3</span><span class="period-time">(10:00 -
10:30)</span></th>
            <th><span class="period-num">4</span><span class="period-time">(10:30 -
11:00)</span></th>
            <th><span class="period-num">5</span><span class="period-time">(11:00 -
11:30)</span></th>
            <th><span class="period-num">6</span><span class="period-time">(11:30 -
12:00)</span></th>
            <th><span class="period-num">7</span><span class="period-time">(12:00 -
12:25)</span></th>
            <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
```

```
      </tr>
    </thead>
    <tbody id="table-body">
      </tbody>
</table>

<script>
    // Full Schedule Data from the image
    const scheduleData = [
        { day: "Sunday", dayIndex: 0, classes: [
            { p: 1, sub: "7A" }, { p: 2, sub: "" }, { p: 3, sub: "4A" }, { p: 4, sub: "" },
            { p: 5, sub: "5B" }, { p: 6, sub: "5A" }, { p: 7, sub: "" }, { p: 8, sub: "" }
        ]},
        { day: "Monday", dayIndex: 1, classes: [
            { p: 1, sub: "7A", nafs: true }, { p: 2, sub: "" }, { p: 3, sub: "" }, { p: 4, sub: "5B" },
            { p: 5, sub: "" }, { p: 6, sub: "4A" }, { p: 7, sub: "7A" }, { p: 8, sub: "5A" }
        ]},
        { day: "Tuesday", dayIndex: 2, classes: [
            { p: 1, sub: "" }, { p: 2, sub: "" }, { p: 3, sub: "5A", nafs: true }, { p: 4, sub: "" },
            { p: 5, sub: "5B" }, { p: 6, sub: "4A" }, { p: 7, sub: "7A" }, { p: 8, sub: "" }
        ]},
        { day: "Wednesday", dayIndex: 3, classes: [
            { p: 1, sub: "" }, { p: 2, sub: "" }, { p: 3, sub: "" }, { p: 4, sub: "5B" },
            { p: 5, sub: "7A" }, { p: 6, sub: "" }, { p: 7, sub: "5A" }, { p: 8, sub: "" }
        ]},
        { day: "Thursday", dayIndex: 4, classes: [
            { p: 1, sub: "" }, { p: 2, sub: "" }, { p: 3, sub: "4A", nafs: true }, { p: 4, sub: "4A" },
            { p: 5, sub: "5B", nafs: true }, { p: 6, sub: "" }, { p: 7, sub: "5A" }, { p: 8, sub: "" }
        ]}
    ];

    const timeSlots = [
        { start: 9*60, end: 9*60+30 }, // P1
        { start: 9*60+30, end: 10*60 }, // P2
        { start: 10*60, end: 10*60+30 }, // P3
        { start: 10*60+30, end: 11*60 }, // P4
        { start: 11*60, end: 11*60+30 }, // P5
        { start: 11*60+30, end: 12*60 }, // P6
        { start: 12*60, end: 12*60+25 }, // P7
        { start: 12*60+25, end: 12*60+50 } // P8
    ];
```

```
  function buildTable() {
    const tbody = document.getElementById('table-body');

    scheduleData.forEach(dayRow => {
      const tr = document.createElement('tr');
      tr.id = `row-day-${dayRow.dayIndex}`;

      // Day Column
      let html = `<td class="day-col">${dayRow.day}</td>`;

      // Class Columns
      dayRow.classes.forEach((c, index) => {
        let colorClass = 'sub-free';
        if (c.sub === '7A') colorClass = 'sub-7A';
        else if (c.sub === '4A') colorClass = 'sub-4A';
        else if (c.sub!== '') colorClass = 'sub-other';

        const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

        html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-$
{index}">
                  ${c.sub}
                  ${nafsHTML}
                </td>`;
      });

      tr.innerHTML = html;
      tbody.appendChild(tr);
    });
  }

  function updateLiveIndicator() {
    const now = new Date();
    const currentDayIndex = now.getDay(); // 0=Sun, 1=Mon...
    const currentMins = now.getHours() * 60 + now.getMinutes();

    // Update Clock
    document.getElementById('clock').innerText =
      now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

    // Clear previous highlights
    document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-
```

```
current-class'));
      document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-
current-day'));

      // Highlight Current Day row
      const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
      if (dayRow) {
         dayRow.querySelector('.day-col').classList.add('is-current-day');
      }

      // Highlight Current Class Cell
      if (currentDayIndex >= 0 && currentDayIndex <= 4) {
         for (let i = 0; i < timeSlots.length; i++) {
            if (currentMins >= timeSlots[i].start && currentMins < timeSlots[i].end) {
               const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);
               if (activeCell) {
                  activeCell.classList.add('is-current-class');
               }
               break; // Found the period, no need to check others
            }
         }
      }
   }

   // Initialize
   buildTable();
   updateLiveIndicator();
   setInterval(updateLiveIndicator, 30000); // Check every 30 seconds
</script>

</body>
</html>
```

What this version does:

1. **The Full Grid:** It draws the complete Sunday-to-Thursday, Period 1-to-8 grid directly on your screen.
2. **Exact Color Mapping:** Classes labeled 7A are automatically painted yellow, 4A are light blue, and the others are white—exactly mirroring your uploaded image.
3. **The Live Highlighter:** Instead of a moving line (which looks messy on a large grid), this page checks the current time. If you are currently in a class,

that specific square on the grid will **pop out, glow red, and display a tiny "LIVE" tag**.

Since the schedule is embedded right into the <script> tag, you can just save this as schedule.html and double-click it to run perfectly anywhere!

json code

Here is the separated code for the **full weekly grid**.

Just like before, you will need two files saved in the same folder. And remember, because we are using an external file, **you must run this through a local web server** (like VS Code Live Server) so your browser doesn't block the fetch request.

1. The JSON File (full_schedule.json)

Create a new file, name it exactly full_schedule.json, and paste this code inside:

```
[
 {
  "day": "Sunday",
  "dayIndex": 0,
  "classes": [
   { "p": 1, "sub": "7A" }, { "p": 2, "sub": "" }, { "p": 3, "sub": "4A" }, { "p": 4, "sub": "" },
   { "p": 5, "sub": "5B" }, { "p": 6, "sub": "5A" }, { "p": 7, "sub": "" }, { "p": 8, "sub": "" }
  ]
 },
 {
  "day": "Monday",
  "dayIndex": 1,
  "classes": [
   { "p": 1, "sub": "7A", "nafs": true }, { "p": 2, "sub": "" }, { "p": 3, "sub": "" }, { "p": 4, "sub":
"5B" },
   { "p": 5, "sub": "" }, { "p": 6, "sub": "4A" }, { "p": 7, "sub": "7A" }, { "p": 8, "sub": "5A" }
  ]
 },
 {
  "day": "Tuesday",
  "dayIndex": 2,
  "classes": [
   { "p": 1, "sub": "" }, { "p": 2, "sub": "" }, { "p": 3, "sub": "5A", "nafs": true }, { "p": 4, "sub":
"" },
   { "p": 5, "sub": "5B" }, { "p": 6, "sub": "4A" }, { "p": 7, "sub": "7A" }, { "p": 8, "sub": "" }
```

```
    ]
  },
  {
    "day": "Wednesday",
    "dayIndex": 3,
    "classes": [
      { "p": 1, "sub": "" }, { "p": 2, "sub": "" }, { "p": 3, "sub": "" }, { "p": 4, "sub": "5B" },
      { "p": 5, "sub": "7A" }, { "p": 6, "sub": "" }, { "p": 7, "sub": "5A" }, { "p": 8, "sub": "" }
    ]
  },
  {
    "day": "Thursday",
    "dayIndex": 4,
    "classes": [
      { "p": 1, "sub": "" }, { "p": 2, "sub": "" }, { "p": 3, "sub": "4A", "nafs": true }, { "p": 4, "sub":
"4A" },
      { "p": 5, "sub": "5B", "nafs": true }, { "p": 6, "sub": "" }, { "p": 7, "sub": "5A" }, { "p": 8,
"sub": "" }
    ]
  }
]
```

---

2. The HTML Page (index.html)

Save this as index.html in the same folder. This code now fetches the data dynamically from the JSON file above.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Full Weekly Schedule</title>
  <style>
    :root {
      --header-bg: #4a4a4a;
      --day-bg: #5a5a5a;
      --day-text: #fce4ce;
      --border-color: #222;
      --7a-color: #ffff00;
      --4a-color: #dbeafe;
      --other-color: #ffffff;
```

```
    --live-glow: #ff2a2a;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f0f2f5;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

h2 { margin: 0; color: #333; }.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

table {
    width: 100%;
    max-width: 1000px;
    border-collapse: collapse;
    background: white;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    border: 2px solid var(--border-color);
}

th, td {
    border: 1px solid var(--border-color);
    text-align: center;
```

```
        padding: 12px 5px;
        width: 11%;
    }

    /* Headers */
    th {
        background-color: var(--header-bg);
        color: white;
        font-size: 0.9rem;
        line-height: 1.4;
    }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }

    /* Day Column */.day-col {
        background-color: var(--day-bg);
        color: var(--day-text);
        font-size: 1.1rem;
        font-weight: bold;
        text-align: left;
        padding-left: 15px;
        width: 12%;
    }

    /* Subjects */.subject-cell {
        font-size: 1.2rem;
        font-weight: 900;
        color: #111;
        position: relative;
        transition: all 0.3s;
    }.nafs-text {
        display: block;
        font-size: 0.7rem;
        font-weight: normal;
        color: #444;
        margin-top: 5px;
    }

    /* Specific Colors from Image */.sub-7A { background-color: var(--7a-color); }.sub-4A
{ background-color: var(--4a-color); }.sub-other { background-color: var(--other-
color); }.sub-free { background-color: white; }

    /* Live Indicator - Highlights the current active class */.is-current-class {
```

```
            box-shadow: inset 0 0 0 4px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.6);
            z-index: 10;
            transform: scale(1.05);
            border-radius: 4px;
        }.is-current-class::after {
            content: 'LIVE';
            position: absolute;
            top: -10px;
            right: -10px;
            background: var(--live-glow);
            color: white;
            font-size: 0.6rem;
            padding: 2px 6px;
            border-radius: 10px;
            font-weight: bold;
            box-shadow: 0 2px 4px rgba(0,0,0,0.3);
        }.is-current-day {
            border-left: 5px solid var(--live-glow);
        }.error-message {
            color: red;
            font-weight: bold;
            padding: 20px;
            background: #ffebe9;
            border: 1px solid red;
            border-radius: 8px;
            margin-top: 20px;
            display: none;
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2>Full Weekly View</h2>
    <div id="clock" class="clock">00:00</div>
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
    <thead>
        <tr>
```

```
        <th>Day</th>
        <th><span class="period-num">1</span><span class="period-time">(9:00 -
9:30)</span></th>
        <th><span class="period-num">2</span><span class="period-time">(9:30 -
10:00)</span></th>
        <th><span class="period-num">3</span><span class="period-time">(10:00 -
10:30)</span></th>
        <th><span class="period-num">4</span><span class="period-time">(10:30 -
11:00)</span></th>
        <th><span class="period-num">5</span><span class="period-time">(11:00 -
11:30)</span></th>
        <th><span class="period-num">6</span><span class="period-time">(11:30 -
12:00)</span></th>
        <th><span class="period-num">7</span><span class="period-time">(12:00 -
12:25)</span></th>
        <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
      </tr>
    </thead>
    <tbody id="table-body">
      </tbody>
</table>

<script>
    let scheduleData = [];

    const timeSlots = [
        { start: 9*60, end: 9*60+30 }, // P1
        { start: 9*60+30, end: 10*60 }, // P2
        { start: 10*60, end: 10*60+30 }, // P3
        { start: 10*60+30, end: 11*60 }, // P4
        { start: 11*60, end: 11*60+30 }, // P5
        { start: 11*60+30, end: 12*60 }, // P6
        { start: 12*60, end: 12*60+25 }, // P7
        { start: 12*60+25, end: 12*60+50 } // P8
    ];

    async function init() {
        try {
            // Fetching data from the external full_schedule.json file
            const response = await fetch('full_schedule.json');
```

```
        if (!response.ok) {
            throw new Error("HTTP error " + response.status);
        }

        scheduleData = await response.json();

        buildTable();
        updateLiveIndicator();
        setInterval(updateLiveIndicator, 30000); // Check every 30 seconds

    } catch (error) {
        console.error("Failed to load full_schedule.json:", error);
        const errorBox = document.getElementById('error-box');
        errorBox.style.display = "block";
        errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you
are running a local web server (like VS Code Live Server) and not just opening the file
directly in the browser.<br><br>Technical error: ${error.message}`;
        document.getElementById('schedule-table').style.display = "none";
    }
}

function buildTable() {
    const tbody = document.getElementById('table-body');
    tbody.innerHTML = ''; // Clear just in case

    scheduleData.forEach(dayRow => {
        const tr = document.createElement('tr');
        tr.id = `row-day-${dayRow.dayIndex}`;

        // Day Column
        let html = `<td class="day-col">${dayRow.day}</td>`;

        // Class Columns
        dayRow.classes.forEach((c, index) => {
            let colorClass = 'sub-free';
            if (c.sub === '7A') colorClass = 'sub-7A';
            else if (c.sub === '4A') colorClass = 'sub-4A';
            else if (c.sub!== '') colorClass = 'sub-other';

            const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

            html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-$
```

```
{index}">
                    ${c.sub}
                    ${nafsHTML}
                  </td>`;
        });

        tr.innerHTML = html;
        tbody.appendChild(tr);
      });
    }

    function updateLiveIndicator() {
      if (scheduleData.length === 0) return; // Don't run if data failed to load

      const now = new Date();
      const currentDayIndex = now.getDay(); // 0=Sun, 1=Mon...
      const currentMins = now.getHours() * 60 + now.getMinutes();

      // Update Clock
      document.getElementById('clock').innerText =
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

      // Clear previous highlights
      document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-
current-class'));
      document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-
current-day'));

      // Highlight Current Day row
      const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
      if (dayRow) {
        dayRow.querySelector('.day-col').classList.add('is-current-day');
      }

      // Highlight Current Class Cell
      if (currentDayIndex >= 0 && currentDayIndex <= 4) {
        for (let i = 0; i < timeSlots.length; i++) {
          if (currentMins >= timeSlots[i].start && currentMins < timeSlots[i].end) {
            const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);
            if (activeCell) {
              activeCell.classList.add('is-current-class');
            }
```

```
            break;
          }
        }
      }
    }

    // Initialize the fetch and rendering
    init();
</script>

</body>
</html>
```

Would you like me to show you how to add an “Edit” feature so you can click a cell and update the JSON data directly from the browser?

json all scadule

Here is the complete JSON for the entire weekly schedule, carefully transcribed from your image.

This matches the format needed for the full_schedule.json file in the full-grid HTML page I provided previously.


```
[
  {
    "day": "Sunday",
    "dayIndex": 0,
    "classes": [
      { "p": 1, "sub": "7A" },
      { "p": 2, "sub": "" },
      { "p": 3, "sub": "4A" },
      { "p": 4, "sub": "" },
      { "p": 5, "sub": "5B" },
      { "p": 6, "sub": "5A" },
      { "p": 7, "sub": "" },
      { "p": 8, "sub": "" }
    ]
  },
  {
    "day": "Monday",
    "dayIndex": 1,
```

```
  "classes": [
    { "p": 1, "sub": "7A", "nafs": true },
    { "p": 2, "sub": "" },
    { "p": 3, "sub": "" },
    { "p": 4, "sub": "5B" },
    { "p": 5, "sub": "" },
    { "p": 6, "sub": "4A" },
    { "p": 7, "sub": "7A" },
    { "p": 8, "sub": "5A" }
  ]
},
{
  "day": "Tuesday",
  "dayIndex": 2,
  "classes": [
    { "p": 1, "sub": "" },
    { "p": 2, "sub": "" },
    { "p": 3, "sub": "5A", "nafs": true },
    { "p": 4, "sub": "" },
    { "p": 5, "sub": "5B" },
    { "p": 6, "sub": "4A" },
    { "p": 7, "sub": "7A" },
    { "p": 8, "sub": "" }
  ]
},
{
  "day": "Wednesday",
  "dayIndex": 3,
  "classes": [
    { "p": 1, "sub": "" },
    { "p": 2, "sub": "" },
    { "p": 3, "sub": "" },
    { "p": 4, "sub": "5B" },
    { "p": 5, "sub": "7A" },
    { "p": 6, "sub": "" },
    { "p": 7, "sub": "5A" },
    { "p": 8, "sub": "" }
  ]
},
{
  "day": "Thursday",
  "dayIndex": 4,
  "classes": [
```

```
    { "p": 1, "sub": "" },
    { "p": 2, "sub": "" },
    { "p": 3, "sub": "4A", "nafs": true },
    { "p": 4, "sub": "4A" },
    { "p": 5, "sub": "5B", "nafs": true },
    { "p": 6, "sub": "" },
    { "p": 7, "sub": "5A" },
    { "p": 8, "sub": "" }
   ]
  }
]
```

Would you like me to add an export button to the HTML page so you can download this JSON file directly from the browser?

add time line indicator in side the active cell

Here is the updated **HTML file** (index.html).

I have added a dynamic red timeline indicator that physically moves down **inside** the active class cell as the period progresses. It calculates exactly how many minutes have passed in that specific period and positions the line using percentages (e.g., if you are 15 minutes into a 30-minute class, the line will sit exactly in the middle of the cell).

You do **not** need to change your full_schedule.json file; it remains exactly the same. Just replace your existing index.html code with this:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Full Weekly Schedule</title>
  <style>
    :root {
      --header-bg: #4a4a4a;
      --day-bg: #5a5a5a;
      --day-text: #fce4ce;
      --border-color: #222;
      --7a-color: #ffff00;
      --4a-color: #dbeafe;
```

```
    --other-color: #ffffff;
    --live-glow: #ff2a2a;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f0f2f5;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

h2 { margin: 0; color: #333; }.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

table {
    width: 100%;
    max-width: 1000px;
    border-collapse: collapse;
    background: white;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    border: 2px solid var(--border-color);
}

th, td {
    border: 1px solid var(--border-color);
```

```
        text-align: center;
        padding: 12px 5px;
        width: 11%;
    }

    /* Headers */
    th {
        background-color: var(--header-bg);
        color: white;
        font-size: 0.9rem;
        line-height: 1.4;
    }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }

    /* Day Column */.day-col {
        background-color: var(--day-bg);
        color: var(--day-text);
        font-size: 1.1rem;
        font-weight: bold;
        text-align: left;
        padding-left: 15px;
        width: 12%;
    }

    /* Subjects */.subject-cell {
        font-size: 1.2rem;
        font-weight: 900;
        color: #111;
        position: relative; /* Important for positioning the timeline inside */
        transition: all 0.3s;
        overflow: hidden; /* Keeps the line cleanly inside the cell borders */
    }.nafs-text {
        display: block;
        font-size: 0.7rem;
        font-weight: normal;
        color: #444;
        margin-top: 5px;
    }

    /* Specific Colors */.sub-7A { background-color: var(--7a-color); }.sub-4A {
background-color: var(--4a-color); }.sub-other { background-color: var(--other-
color); }.sub-free { background-color: white; }
```

```
    /* Highlight Current Class */.is-current-class {
        box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4);
        z-index: 10;
    }.is-current-class::after {
        content: 'LIVE';
        position: absolute;
        top: 2px;
        right: 2px;
        background: var(--live-glow);
        color: white;
        font-size: 0.6rem;
        padding: 2px 5px;
        border-radius: 4px;
        font-weight: bold;
        z-index: 30; /* Keep text above the moving line */
    }.is-current-day {
        border-left: 5px solid var(--live-glow);
    }

    /* --- THE NEW TIMELINE INDICATOR INSIDE THE CELL --- */.cell-timeline {
        position: absolute;
        left: 0;
        right: 0;
        height: 3px;
        background: linear-gradient(90deg, var(--live-glow) 0%, #ff7b7b 70%, transparent
100%);
        z-index: 20;
        pointer-events: none;
        transition: top 0.5s linear;
        display: none; /* Hidden by default */
    }.cell-timeline::before {
        content: '';
        position: absolute;
        left: 0;
        top: -4px;
        width: 10px;
        height: 10px;
        background: var(--live-glow);
        border-radius: 50%;
        border: 2px solid white;
        box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
    }.error-message {
```

```
            color: red;
            font-weight: bold;
            padding: 20px;
            background: #ffebe9;
            border: 1px solid red;
            border-radius: 8px;
            margin-top: 20px;
            display: none;
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2>Full Weekly View</h2>
    <div id="clock" class="clock">00:00</div>
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
    <thead>
        <tr>
            <th>Day</th>
            <th><span class="period-num">1</span><span class="period-time">(9:00 -
9:30)</span></th>
            <th><span class="period-num">2</span><span class="period-time">(9:30 -
10:00)</span></th>
            <th><span class="period-num">3</span><span class="period-time">(10:00 -
10:30)</span></th>
            <th><span class="period-num">4</span><span class="period-time">(10:30 -
11:00)</span></th>
            <th><span class="period-num">5</span><span class="period-time">(11:00 -
11:30)</span></th>
            <th><span class="period-num">6</span><span class="period-time">(11:30 -
12:00)</span></th>
            <th><span class="period-num">7</span><span class="period-time">(12:00 -
12:25)</span></th>
            <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
        </tr>
    </thead>
```

```
    <tbody id="table-body">
    </tbody>
</table>

<script>
    let scheduleData = [];

    const timeSlots = [
        { start: 9*60, end: 9*60+30 }, // P1
        { start: 9*60+30, end: 10*60 }, // P2
        { start: 10*60, end: 10*60+30 }, // P3
        { start: 10*60+30, end: 11*60 }, // P4
        { start: 11*60, end: 11*60+30 }, // P5
        { start: 11*60+30, end: 12*60 }, // P6
        { start: 12*60, end: 12*60+25 }, // P7
        { start: 12*60+25, end: 12*60+50 } // P8
    ];

    // Create the timeline element once in memory
    const timelineIndicator = document.createElement('div');
    timelineIndicator.className = 'cell-timeline';
    timelineIndicator.id = 'active-timeline';

    async function init() {
        try {
            const response = await fetch('full_schedule.json');
            if (!response.ok) throw new Error("HTTP error " + response.status);

            scheduleData = await response.json();

            buildTable();
            updateLiveIndicator();
            // Check every 10 seconds for smoother line movement
            setInterval(updateLiveIndicator, 10000);

        } catch (error) {
            console.error("Failed to load full_schedule.json:", error);
            const errorBox = document.getElementById('error-box');
            errorBox.style.display = "block";
            errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you
are running a local web server (like VS Code Live Server).`;
            document.getElementById('schedule-table').style.display = "none";
```

```
    }
  }

  function buildTable() {
    const tbody = document.getElementById('table-body');
    tbody.innerHTML = '';

    scheduleData.forEach(dayRow => {
      const tr = document.createElement('tr');
      tr.id = `row-day-${dayRow.dayIndex}`;

      let html = `<td class="day-col">${dayRow.day}</td>`;

      dayRow.classes.forEach((c, index) => {
        let colorClass = 'sub-free';
        if (c.sub === '7A') colorClass = 'sub-7A';
        else if (c.sub === '4A') colorClass = 'sub-4A';
        else if (c.sub!== '') colorClass = 'sub-other';

        const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

        html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-$
{index}">
                  ${c.sub}
                  ${nafsHTML}
                </td>`;
      });

      tr.innerHTML = html;
      tbody.appendChild(tr);
    });
  }

  function updateLiveIndicator() {
    if (scheduleData.length === 0) return;

    const now = new Date();
    const currentDayIndex = now.getDay();
    const currentMins = now.getHours() * 60 + now.getMinutes();

    document.getElementById('clock').innerText =
      now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
```

```
    // Reset previous highlights
    document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-current-class'));
    document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-current-day'));
    timelineIndicator.style.display = 'none'; // Hide line initially

    // Highlight Current Day row
    const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
    if (dayRow) {
      dayRow.querySelector('.day-col').classList.add('is-current-day');
    }

    // Highlight Current Class Cell AND inject the timeline
    let isClassActive = false;

    if (currentDayIndex >= 0 && currentDayIndex <= 4) {
      for (let i = 0; i < timeSlots.length; i++) {
        if (currentMins >= timeSlots[i].start && currentMins < timeSlots[i].end) {
          const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);

          if (activeCell) {
            activeCell.classList.add('is-current-class');

            // Calculate percentage of time passed inside this specific period
            const slotDuration = timeSlots[i].end - timeSlots[i].start;
            const minsPassed = currentMins - timeSlots[i].start;
            const percentComplete = (minsPassed / slotDuration) * 100;

            // Move timeline into this cell and position it
            activeCell.appendChild(timelineIndicator);
            timelineIndicator.style.display = 'block';
            timelineIndicator.style.top = `${percentComplete}%`;

            isClassActive = true;
          }
          break;
        }
      }
    }
  }
```

```
    init();
</script>

</body>
</html>
```

How the new feature works:

1. **The Injection Mechanism:** The JavaScript creates a single glowing line (timelineIndicator) in memory.
2. **Dynamic Parenting:** When the script detects which class is currently active, it physically moves that line's HTML element inside the active <td> cell.
3. **Percentage Math:** It checks the duration of the current period (e.g., 25 minutes for Period 7), checks how many minutes have passed, turns that into a percentage (e.g., 50%), and assigns top: 50% to the line. This pushes the line down the cell perfectly in real-time.

add dtn to show all days or current only

Here is the updated index.html code. I have added a new "View" button next to the clock in the top right.

When you click it, the table instantly filters to show **only today's row**. Click it again, and the **full week** slides back into view. The timeline indicator continues to work perfectly in both modes!

(Note: Your full_schedule.json stays exactly the same. You only need to update the HTML file.)

Updated HTML Page (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Full Weekly Schedule</title>
    <style>
        :root {
            --header-bg: #4a4a4a;
            --day-bg: #5a5a5a;
```

```
    --day-text: #fce4ce;
    --border-color: #222;
    --7a-color: #ffff00;
    --4a-color: #dbeafe;
    --other-color: #ffffff;
    --live-glow: #ff2a2a;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f0f2f5;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}.header-title {
    margin: 0;
    color: #333;
}.controls {
    display: flex;
    gap: 15px;
    align-items: center;
}

/* New Button Styles */.toggle-btn {
    padding: 8px 16px;
    background: linear-gradient(135deg, #10b981 0%, #059669 100%);
    color: white;
    border: none;
    border-radius: 8px;
    font-weight: bold;
    font-size: 0.95rem;
    cursor: pointer;
    box-shadow: 0 4px 6px rgba(16, 185, 129, 0.2), inset 0 2px 0 rgba(255,255,255,0.2);
    transition: all 0.2s;
```

```
    }.toggle-btn:hover {
        transform: translateY(-1px);
        box-shadow: 0 6px 8px rgba(16, 185, 129, 0.3);
    }.clock {
        font-size: 1.2rem;
        font-weight: bold;
        color: #3b82f6;
        background: white;
        padding: 8px 16px;
        border-radius: 8px;
        box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }

    table {
        width: 100%;
        max-width: 1000px;
        border-collapse: collapse;
        background: white;
        box-shadow: 0 10px 30px rgba(0,0,0,0.1);
        border: 2px solid var(--border-color);
        transition: all 0.3s ease;
    }

    th, td {
        border: 1px solid var(--border-color);
        text-align: center;
        padding: 12px 5px;
        width: 11%;
    }

    /* Headers */
    th {
        background-color: var(--header-bg);
        color: white;
        font-size: 0.9rem;
        line-height: 1.4;
    }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }

    /* Day Column */.day-col {
        background-color: var(--day-bg);
        color: var(--day-text);
```

```
    font-size: 1.1rem;
    font-weight: bold;
    text-align: left;
    padding-left: 15px;
    width: 12%;
}

/* Subjects */.subject-cell {
    font-size: 1.2rem;
    font-weight: 900;
    color: #111;
    position: relative;
    transition: all 0.3s;
    overflow: hidden;
}.nafs-text {
    display: block;
    font-size: 0.7rem;
    font-weight: normal;
    color: #444;
    margin-top: 5px;
    position: relative;
    z-index: 5;
}.subject-cell span.main-text {
    position: relative;
    z-index: 5;
}

/* Specific Colors from Image */.sub-7A { background-color: var(--7a-color); }.sub-4A { background-color: var(--4a-color); }.sub-other { background-color: var(--other-color); }.sub-free { background-color: white; }

/* Live Indicator Styling */.is-current-class {
    box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4);
    z-index: 10;
}.is-current-day {
    border-left: 5px solid var(--live-glow);
}.cell-progress-line {
    position: absolute;
    left: 0;
    right: 0;
    height: 3px;
    background: rgba(255, 42, 42, 0.8);
```

```
            z-index: 1;
            pointer-events: none;
            box-shadow: 0 0 8px rgba(255, 42, 42, 0.6);
            transition: top 0.3s linear;
        }.cell-progress-line::before {
            content: '';
            position: absolute;
            left: 0;
            top: -4px;
            width: 10px;
            height: 10px;
            background: var(--live-glow);
            border-radius: 50%;
            box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
        }.cell-progress-fill {
            position: absolute;
            top: 0;
            left: 0;
            right: 0;
            background: rgba(255, 42, 42, 0.1);
            z-index: 0;
            pointer-events: none;
        }.error-message {
            color: red;
            font-weight: bold;
            padding: 20px;
            background: #ffebe9;
            border: 1px solid red;
            border-radius: 8px;
            margin-top: 20px;
            display: none;
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2 class="header-title" id="main-title">Full Weekly View</h2>
    <div class="controls">
        <button id="toggle-view-btn" class="toggle-btn" onclick="toggleViewMode()">Show
Today Only</button>
        <div id="clock" class="clock">00:00</div>
    </div>
```

```
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
  <thead>
    <tr>
      <th>Day</th>
      <th><span class="period-num">1</span><span class="period-time">(9:00 - 9:30)</span></th>
      <th><span class="period-num">2</span><span class="period-time">(9:30 - 10:00)</span></th>
      <th><span class="period-num">3</span><span class="period-time">(10:00 - 10:30)</span></th>
      <th><span class="period-num">4</span><span class="period-time">(10:30 - 11:00)</span></th>
      <th><span class="period-num">5</span><span class="period-time">(11:00 - 11:30)</span></th>
      <th><span class="period-num">6</span><span class="period-time">(11:30 - 12:00)</span></th>
      <th><span class="period-num">7</span><span class="period-time">(12:00 - 12:25)</span></th>
      <th><span class="period-num">8</span><span class="period-time">(12:25 - 12:50)</span></th>
    </tr>
  </thead>
  <tbody id="table-body">
    </tbody>
</table>

<script>
  let scheduleData = [];
  let isShowingAllDays = true; // State tracker for the view mode

  const timeSlots = [
    { start: 9*60, end: 9*60+30 }, // P1
    { start: 9*60+30, end: 10*60 }, // P2
    { start: 10*60, end: 10*60+30 }, // P3
    { start: 10*60+30, end: 11*60 }, // P4
    { start: 11*60, end: 11*60+30 }, // P5
    { start: 11*60+30, end: 12*60 }, // P6
    { start: 12*60, end: 12*60+25 }, // P7
```

```
        { start: 12*60+25, end: 12*60+50 } // P8
    ];

    async function init() {
        try {
            const response = await fetch('full_schedule.json');
            if (!response.ok) throw new Error("HTTP error " + response.status);

            scheduleData = await response.json();

            buildTable();
            updateLiveIndicator();
            setInterval(updateLiveIndicator, 30000); // Check every 30 seconds

        } catch (error) {
            console.error("Failed to load full_schedule.json:", error);
            const errorBox = document.getElementById('error-box');
            errorBox.style.display = "block";
            errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you are running a local web server (like VS Code Live Server) and not just opening the file directly in the browser.<br><br>Technical error: ${error.message}`;
            document.getElementById('schedule-table').style.display = "none";
        }
    }

    function buildTable() {
        const tbody = document.getElementById('table-body');
        tbody.innerHTML = '';

        scheduleData.forEach(dayRow => {
            const tr = document.createElement('tr');
            tr.id = `row-day-${dayRow.dayIndex}`;

            let html = `<td class="day-col">${dayRow.day}</td>`;

            dayRow.classes.forEach((c, index) => {
                let colorClass = 'sub-free';
                if (c.sub === '7A') colorClass = 'sub-7A';
                else if (c.sub === '4A') colorClass = 'sub-4A';
                else if (c.sub!== '') colorClass = 'sub-other';

                const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';
```

```
            html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-${index}">
                        <span class="main-text">${c.sub}</span>
                        ${nafsHTML}
                    </td>`;
        });

        tr.innerHTML = html;
        tbody.appendChild(tr);
    });

    // Apply view filter just in case
    applyViewFilter();
}

// --- NEW LOGIC FOR TOGGLING VIEWS ---
function toggleViewMode() {
    isShowingAllDays =!isShowingAllDays;

    const btn = document.getElementById('toggle-view-btn');
    const title = document.getElementById('main-title');

    if (isShowingAllDays) {
        btn.innerText = "Show Today Only";
        title.innerText = "Full Weekly View";
    } else {
        btn.innerText = "Show All Days";
        title.innerText = "Today's View";
    }

    applyViewFilter();
}

function applyViewFilter() {
    let currentDayIndex = new Date().getDay();
    // If it's Friday (5) or Saturday (6), default to Sunday (0) so the screen isn't completely blank
    if (currentDayIndex > 4) currentDayIndex = 0;

    scheduleData.forEach(dayRow => {
        const tr = document.getElementById(`row-day-${dayRow.dayIndex}`);
```

```
        if (!tr) return;

        if (isShowingAllDays) {
            tr.style.display = ''; // Show all rows
        } else {
            // Hide row if it doesn't match today's index
            if (dayRow.dayIndex === currentDayIndex) {
                tr.style.display = '';
            } else {
                tr.style.display = 'none';
            }
        }
    });
}

// --- LIVE INDICATOR LOGIC ---
function updateLiveIndicator() {
    if (scheduleData.length === 0) return;

    const now = new Date();
    const currentDayIndex = now.getDay();
    const currentMins = now.getHours() * 60 + now.getMinutes();

    document.getElementById('clock').innerText =
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

    document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-current-class'));
    document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-current-day'));
    document.querySelectorAll('.cell-progress-line,.cell-progress-fill').forEach(el => el.remove());

    const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
    if (dayRow) {
        dayRow.querySelector('.day-col').classList.add('is-current-day');
    }

    if (currentDayIndex >= 0 && currentDayIndex <= 4) {
        for (let i = 0; i < timeSlots.length; i++) {
            if (currentMins >= timeSlots[i].start && currentMins < timeSlots[i].end) {
                const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);
```

```
            if (activeCell) {
              activeCell.classList.add('is-current-class');

              const totalDuration = timeSlots[i].end - timeSlots[i].start;
              const elapsed = currentMins - timeSlots[i].start;
              const percentDone = (elapsed / totalDuration) * 100;

              const line = document.createElement('div');
              line.className = 'cell-progress-line';
              line.style.top = `${percentDone}%`;

              const fill = document.createElement('div');
              fill.className = 'cell-progress-fill';
              fill.style.height = `${percentDone}%`;

              activeCell.appendChild(fill);
              activeCell.appendChild(line);
            }
            break;
          }
        }
      }
    }

    init();
</script>

</body>
</html>
```

What I Added:

- **Toggle Button:** Added the <button id="toggle-view-btn"> in the header next to the clock.
- **toggleViewMode() function:** When clicked, this switches between tracking the entire week vs. just your current day.
- **applyViewFilter() function:** Automatically hides the other rows via CSS display: none; without breaking the grid structure or losing your table headers.

- **Weekend Safety:** I programmed it so that if you click “Show Today Only” on a Friday or Saturday, it smartly defaults to showing Sunday’s row instead of a completely empty screen.

Would you like me to make it so the page remembers your choice (All Days vs Today) even after you refresh your browser?

add time left for current period

Here is the updated index.html. I have added a sleek **“Time Left” badge** that appears in the bottom corner of your current active class.

To make this feature really shine, I upgraded the page’s refresh rate from 30 seconds to **every 1 second**. This means the countdown will tick down live (e.g., 14:59 left), and the red progress line will slide down the cell smoothly in real-time.

Updated HTML Page (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Full Weekly Schedule</title>
  <style>
    :root {
      --header-bg: #4a4a4a;
      --day-bg: #5a5a5a;
      --day-text: #fce4ce;
      --border-color: #222;
      --7a-color: #ffff00;
      --4a-color: #dbeafe;
      --other-color: #ffffff;
      --live-glow: #ff2a2a;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: #f0f2f5;
      display: flex;
      flex-direction: column;
      align-items: center;
```

```
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}.header-title {
    margin: 0;
    color: #333;
}.controls {
    display: flex;
    gap: 15px;
    align-items: center;
}.toggle-btn {
    padding: 8px 16px;
    background: linear-gradient(135deg, #10b981 0%, #059669 100%);
    color: white;
    border: none;
    border-radius: 8px;
    font-weight: bold;
    font-size: 0.95rem;
    cursor: pointer;
    box-shadow: 0 4px 6px rgba(16, 185, 129, 0.2), inset 0 2px 0 rgba(255,255,255,0.2);
    transition: all 0.2s;
}.toggle-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 6px 8px rgba(16, 185, 129, 0.3);
}.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

table {
    width: 100%;
    max-width: 1000px;
```

```
            border-collapse: collapse;
            background: white;
            box-shadow: 0 10px 30px rgba(0,0,0,0.1);
            border: 2px solid var(--border-color);
            transition: all 0.3s ease;
        }

        th, td {
            border: 1px solid var(--border-color);
            text-align: center;
            padding: 12px 5px;
            width: 11%;
        }

        th {
            background-color: var(--header-bg);
            color: white;
            font-size: 0.9rem;
            line-height: 1.4;
        }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }.day-col {
            background-color: var(--day-bg);
            color: var(--day-text);
            font-size: 1.1rem;
            font-weight: bold;
            text-align: left;
            padding-left: 15px;
            width: 12%;
        }.subject-cell {
            font-size: 1.2rem;
            font-weight: 900;
            color: #111;
            position: relative;
            transition: all 0.3s;
            overflow: hidden;
        }.nafs-text {
            display: block;
            font-size: 0.7rem;
            font-weight: normal;
            color: #444;
            margin-top: 5px;
            position: relative;
            z-index: 5;
```

```
}.subject-cell span.main-text {
    position: relative;
    z-index: 5;
}.sub-7A { background-color: var(--7a-color); }.sub-4A { background-color: var(--4a-color); }.sub-other { background-color: var(--other-color); }.sub-free { background-color: white; }.is-current-class {
    box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4);
    z-index: 10;
}.is-current-day {
    border-left: 5px solid var(--live-glow);
}.cell-progress-line {
    position: absolute;
    left: 0;
    right: 0;
    height: 3px;
    background: rgba(255, 42, 42, 0.8);
    z-index: 1;
    pointer-events: none;
    box-shadow: 0 0 8px rgba(255, 42, 42, 0.6);
}.cell-progress-line::before {
    content: '';
    position: absolute;
    left: 0;
    top: -4px;
    width: 10px;
    height: 10px;
    background: var(--live-glow);
    border-radius: 50%;
    box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
}.cell-progress-fill {
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    background: rgba(255, 42, 42, 0.1);
    z-index: 0;
    pointer-events: none;
}

/* NEW: Time Left Badge styling */.time-left-badge {
    position: absolute;
    bottom: 4px;
    right: 4px;
```

```
            background: var(--live-glow);
            color: white;
            font-size: 0.7rem;
            padding: 3px 6px;
            border-radius: 4px;
            font-weight: bold;
            z-index: 15;
            box-shadow: 0 2px 4px rgba(0,0,0,0.3);
            letter-spacing: 0.5px;
        }.error-message {
            color: red;
            font-weight: bold;
            padding: 20px;
            background: #ffebe9;
            border: 1px solid red;
            border-radius: 8px;
            margin-top: 20px;
            display: none;
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2 class="header-title" id="main-title">Full Weekly View</h2>
    <div class="controls">
        <button id="toggle-view-btn" class="toggle-btn" onclick="toggleViewMode()">Show
Today Only</button>
        <div id="clock" class="clock">00:00:00</div>
    </div>
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
    <thead>
        <tr>
            <th>Day</th>
            <th><span class="period-num">1</span><span class="period-time">(9:00 -
9:30)</span></th>
            <th><span class="period-num">2</span><span class="period-time">(9:30 -
10:00)</span></th>
```

```
        <th><span class="period-num">3</span><span class="period-time">(10:00 -
10:30)</span></th>
        <th><span class="period-num">4</span><span class="period-time">(10:30 -
11:00)</span></th>
        <th><span class="period-num">5</span><span class="period-time">(11:00 -
11:30)</span></th>
        <th><span class="period-num">6</span><span class="period-time">(11:30 -
12:00)</span></th>
        <th><span class="period-num">7</span><span class="period-time">(12:00 -
12:25)</span></th>
        <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
      </tr>
    </thead>
    <tbody id="table-body">
    </tbody>
</table>

<script>
    let scheduleData = [];
    let isShowingAllDays = true;

    // Time slots stored in total minutes from midnight
    const timeSlots = [
        { start: 9*60, end: 9*60+30 },
        { start: 9*60+30, end: 10*60 },
        { start: 10*60, end: 10*60+30 },
        { start: 10*60+30, end: 11*60 },
        { start: 11*60, end: 11*60+30 },
        { start: 11*60+30, end: 12*60 },
        { start: 12*60, end: 12*60+25 },
        { start: 12*60+25, end: 12*60+50 }
    ];

    async function init() {
        try {
            const response = await fetch('full_schedule.json');
            if (!response.ok) throw new Error("HTTP error " + response.status);

            scheduleData = await response.json();

            buildTable();
```

```
        updateLiveIndicator();
        // Upgraded interval to 1000ms (1 second) for live countdowns
        setInterval(updateLiveIndicator, 1000);

      } catch (error) {
        console.error("Failed to load:", error);
        const errorBox = document.getElementById('error-box');
        errorBox.style.display = "block";
        errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you
are using a local web server.`;
        document.getElementById('schedule-table').style.display = "none";
      }
    }

    function buildTable() {
      const tbody = document.getElementById('table-body');
      tbody.innerHTML = '';

      scheduleData.forEach(dayRow => {
        const tr = document.createElement('tr');
        tr.id = `row-day-${dayRow.dayIndex}`;

        let html = `<td class="day-col">${dayRow.day}</td>`;

        dayRow.classes.forEach((c, index) => {
          let colorClass = 'sub-free';
          if (c.sub === '7A') colorClass = 'sub-7A';
          else if (c.sub === '4A') colorClass = 'sub-4A';
          else if (c.sub!== '') colorClass = 'sub-other';

          const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

          html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-$
{index}">
                    <span class="main-text">${c.sub}</span>
                    ${nafsHTML}
                  </td>`;
        });

        tr.innerHTML = html;
        tbody.appendChild(tr);
      });
```

```
    applyViewFilter();
}

function toggleViewMode() {
    isShowingAllDays =!isShowingAllDays;

    const btn = document.getElementById('toggle-view-btn');
    const title = document.getElementById('main-title');

    if (isShowingAllDays) {
        btn.innerText = "Show Today Only";
        title.innerText = "Full Weekly View";
    } else {
        btn.innerText = "Show All Days";
        title.innerText = "Today's View";
    }
    applyViewFilter();
}

function applyViewFilter() {
    let currentDayIndex = new Date().getDay();
    if (currentDayIndex > 4) currentDayIndex = 0;

    scheduleData.forEach(dayRow => {
        const tr = document.getElementById(`row-day-${dayRow.dayIndex}`);
        if (!tr) return;

        if (isShowingAllDays) {
            tr.style.display = '';
        } else {
            tr.style.display = (dayRow.dayIndex === currentDayIndex)? '' : 'none';
        }
    });
}

function updateLiveIndicator() {
    if (scheduleData.length === 0) return;

    const now = new Date();
    const currentDayIndex = now.getDay();
```

```
    // Use Seconds for highly accurate positioning and countdowns
    const currentTotalSecs = now.getHours() * 3600 + now.getMinutes() * 60 +
now.getSeconds();

    document.getElementById('clock').innerText =
      now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' });

    // Cleanup old elements
    document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-
current-class'));
    document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-
current-day'));
    document.querySelectorAll('.cell-progress-line,.cell-progress-fill,.time-left-
badge').forEach(el => el.remove());

    const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
    if (dayRow) {
      dayRow.querySelector('.day-col').classList.add('is-current-day');
    }

    if (currentDayIndex >= 0 && currentDayIndex <= 4) {
      for (let i = 0; i < timeSlots.length; i++) {

        const startSecs = timeSlots[i].start * 60;
        const endSecs = timeSlots[i].end * 60;

        if (currentTotalSecs >= startSecs && currentTotalSecs < endSecs) {
          const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);

          if (activeCell) {
            activeCell.classList.add('is-current-class');

            // 1. Calculate percentage for the moving line
            const totalDurationSecs = endSecs - startSecs;
            const elapsedSecs = currentTotalSecs - startSecs;
            const percentDone = (elapsedSecs / totalDurationSecs) * 100;

            // 2. Calculate remaining time
            const timeRemainingSecs = endSecs - currentTotalSecs;
            const minsLeft = Math.floor(timeRemainingSecs / 60);
            const secsLeft = timeRemainingSecs % 60;
            const formattedTimeLeft = `${minsLeft}:${secsLeft.toString().padStart(2,
```

```
'0')}`;

                // Inject the moving line
                const line = document.createElement('div');
                line.className = 'cell-progress-line';
                line.style.top = `${percentDone}%`;

                // Inject the background fill
                const fill = document.createElement('div');
                fill.className = 'cell-progress-fill';
                fill.style.height = `${percentDone}%`;

                // Inject the Time Left Badge
                const badge = document.createElement('div');
                badge.className = 'time-left-badge';
                badge.innerText = `${formattedTimeLeft} left`;

                activeCell.appendChild(fill);
                activeCell.appendChild(line);
                activeCell.appendChild(badge);
              }
              break;
            }
          }
        }
      }

  init();
</script>

</body>
</html>
```

What changed:

- **The Clock Includes Seconds:** The digital clock in the header now ticks every second so you can visually verify the page is updating.
- **Live Countdown Badge:** Added a .time-left-badge inside the active period's cell. It sits cleanly in the bottom right corner showing MM:SS left (e.g., "14:45 left").
- **Smoother Animations:** Because the script now checks the time every 1000 milliseconds (1 second) instead of every 30 seconds, the red line indicator

will drift downwards much more smoothly over the duration of the 30-minute class!

“Time Left” move it outside with the current time

Moving the countdown to the header alongside the main clock is a great design choice—it keeps your schedule grid clean and makes the remaining time much easier to read from a distance.

Here is the updated index.html. The countdown has been completely removed from the inside of the cell. It now lives right below the digital clock in the top right corner, complete with a subtle red pulsing effect to draw attention when a class is actively running!

(You do not need to change your full_schedule.json file. Just replace the code in index.html.)

Updated HTML Page (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Full Weekly Schedule</title>
  <style>
    :root {
      --header-bg: #4a4a4a;
      --day-bg: #5a5a5a;
      --day-text: #fce4ce;
      --border-color: #222;
      --7a-color: #ffff00;
      --4a-color: #dbeafe;
      --other-color: #ffffff;
      --live-glow: #ff2a2a;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: #f0f2f5;
      display: flex;
      flex-direction: column;
      align-items: center;
```

```
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}.header-title {
    margin: 0;
    color: #333;
}.controls {
    display: flex;
    gap: 20px;
    align-items: flex-start;
}.toggle-btn {
    padding: 10px 16px;
    margin-top: 2px;
    background: linear-gradient(135deg, #10b981 0%, #059669 100%);
    color: white;
    border: none;
    border-radius: 8px;
    font-weight: bold;
    font-size: 0.95rem;
    cursor: pointer;
    box-shadow: 0 4px 6px rgba(16, 185, 129, 0.2), inset 0 2px 0 rgba(255,255,255,0.2);
    transition: all 0.2s;
}.toggle-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 6px 8px rgba(16, 185, 129, 0.3);
}

/* Grouping the clock and the time left indicator */.time-wrapper {
    display: flex;
    flex-direction: column;
    align-items: flex-end;
    gap: 6px;
}.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
```

```
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

/* NEW: Header Time Left Badge */.header-time-left {
    font-size: 0.85rem;
    font-weight: bold;
    color: var(--live-glow);
    background: #ffebe9;
    padding: 4px 12px;
    border-radius: 6px;
    border: 1px solid rgba(255, 42, 42, 0.2);
    display: none; /* Hidden by default until an active class is found */
    animation: pulse 2s infinite;
}

@keyframes pulse {
    0% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0.4); }
    70% { box-shadow: 0 0 0 6px rgba(255, 42, 42, 0); }
    100% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0); }
}

table {
    width: 100%;
    max-width: 1000px;
    border-collapse: collapse;
    background: white;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    border: 2px solid var(--border-color);
    transition: all 0.3s ease;
}

th, td {
    border: 1px solid var(--border-color);
    text-align: center;
    padding: 12px 5px;
    width: 11%;
}

th {
    background-color: var(--header-bg);
```

```
            color: white;
            font-size: 0.9rem;
            line-height: 1.4;
        }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }.day-col {
            background-color: var(--day-bg);
            color: var(--day-text);
            font-size: 1.1rem;
            font-weight: bold;
            text-align: left;
            padding-left: 15px;
            width: 12%;
        }.subject-cell {
            font-size: 1.2rem;
            font-weight: 900;
            color: #111;
            position: relative;
            transition: all 0.3s;
            overflow: hidden;
        }.nafs-text {
            display: block;
            font-size: 0.7rem;
            font-weight: normal;
            color: #444;
            margin-top: 5px;
            position: relative;
            z-index: 5;
        }.subject-cell span.main-text {
            position: relative;
            z-index: 5;
        }.sub-7A { background-color: var(--7a-color); }.sub-4A { background-color: var(--4a-
color); }.sub-other { background-color: var(--other-color); }.sub-free { background-
color: white; }.is-current-class {
            box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4);
            z-index: 10;
        }.is-current-day {
            border-left: 5px solid var(--live-glow);
        }.cell-progress-line {
            position: absolute;
            left: 0;
            right: 0;
            height: 3px;
            background: rgba(255, 42, 42, 0.8);
```

```
            z-index: 1;
            pointer-events: none;
            box-shadow: 0 0 8px rgba(255, 42, 42, 0.6);
        }.cell-progress-line::before {
            content: '';
            position: absolute;
            left: 0;
            top: -4px;
            width: 10px;
            height: 10px;
            background: var(--live-glow);
            border-radius: 50%;
            box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
        }.cell-progress-fill {
            position: absolute;
            top: 0;
            left: 0;
            right: 0;
            background: rgba(255, 42, 42, 0.1);
            z-index: 0;
            pointer-events: none;
        }.error-message {
            color: red;
            font-weight: bold;
            padding: 20px;
            background: #ffebe9;
            border: 1px solid red;
            border-radius: 8px;
            margin-top: 20px;
            display: none;
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2 class="header-title" id="main-title">Full Weekly View</h2>
    <div class="controls">
        <button id="toggle-view-btn" class="toggle-btn" onclick="toggleViewMode()">Show
Today Only</button>
        <div class="time-wrapper">
            <div id="clock" class="clock">00:00:00</div>
            <div id="header-time-left" class="header-time-left"></div>
```

```
      </div>
    </div>
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
    <thead>
      <tr>
        <th>Day</th>
        <th><span class="period-num">1</span><span class="period-time">(9:00 -
9:30)</span></th>
        <th><span class="period-num">2</span><span class="period-time">(9:30 -
10:00)</span></th>
        <th><span class="period-num">3</span><span class="period-time">(10:00 -
10:30)</span></th>
        <th><span class="period-num">4</span><span class="period-time">(10:30 -
11:00)</span></th>
        <th><span class="period-num">5</span><span class="period-time">(11:00 -
11:30)</span></th>
        <th><span class="period-num">6</span><span class="period-time">(11:30 -
12:00)</span></th>
        <th><span class="period-num">7</span><span class="period-time">(12:00 -
12:25)</span></th>
        <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
      </tr>
    </thead>
    <tbody id="table-body">
    </tbody>
</table>

<script>
    let scheduleData = [];
    let isShowingAllDays = true;

    // Time slots stored in total minutes from midnight
    const timeSlots = [
        { start: 9*60, end: 9*60+30 },
        { start: 9*60+30, end: 10*60 },
        { start: 10*60, end: 10*60+30 },
        { start: 10*60+30, end: 11*60 },
```

```
        { start: 11*60, end: 11*60+30 },
        { start: 11*60+30, end: 12*60 },
        { start: 12*60, end: 12*60+25 },
        { start: 12*60+25, end: 12*60+50 }
    ];

    async function init() {
        try {
            const response = await fetch('full_schedule.json');
            if (!response.ok) throw new Error("HTTP error " + response.status);

            scheduleData = await response.json();

            buildTable();
            updateLiveIndicator();
            setInterval(updateLiveIndicator, 1000);

        } catch (error) {
            console.error("Failed to load:", error);
            const errorBox = document.getElementById('error-box');
            errorBox.style.display = "block";
            errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you
are using a local web server.`;
            document.getElementById('schedule-table').style.display = "none";
        }
    }

    function buildTable() {
        const tbody = document.getElementById('table-body');
        tbody.innerHTML = '';

        scheduleData.forEach(dayRow => {
            const tr = document.createElement('tr');
            tr.id = `row-day-${dayRow.dayIndex}`;

            let html = `<td class="day-col">${dayRow.day}</td>`;

            dayRow.classes.forEach((c, index) => {
                let colorClass = 'sub-free';
                if (c.sub === '7A') colorClass = 'sub-7A';
                else if (c.sub === '4A') colorClass = 'sub-4A';
                else if (c.sub!== '') colorClass = 'sub-other';
```

```
            const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

            html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-${index}">
                        <span class="main-text">${c.sub}</span>
                        ${nafsHTML}
                    </td>`;
        });

        tr.innerHTML = html;
        tbody.appendChild(tr);
    });

    applyViewFilter();
}

function toggleViewMode() {
    isShowingAllDays =!isShowingAllDays;

    const btn = document.getElementById('toggle-view-btn');
    const title = document.getElementById('main-title');

    if (isShowingAllDays) {
        btn.innerText = "Show Today Only";
        title.innerText = "Full Weekly View";
    } else {
        btn.innerText = "Show All Days";
        title.innerText = "Today's View";
    }
    applyViewFilter();
}

function applyViewFilter() {
    let currentDayIndex = new Date().getDay();
    if (currentDayIndex > 4) currentDayIndex = 0;

    scheduleData.forEach(dayRow => {
        const tr = document.getElementById(`row-day-${dayRow.dayIndex}`);
        if (!tr) return;

        if (isShowingAllDays) {
```

```
            tr.style.display = '';
        } else {
            tr.style.display = (dayRow.dayIndex === currentDayIndex)? '' : 'none';
        }
    });
}

function updateLiveIndicator() {
    if (scheduleData.length === 0) return;

    const now = new Date();
    const currentDayIndex = now.getDay();

    const currentTotalSecs = now.getHours() * 3600 + now.getMinutes() * 60 +
now.getSeconds();

    document.getElementById('clock').innerText =
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' });

    // Cleanup grid elements
    document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-
current-class'));
    document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-
current-day'));
    document.querySelectorAll('.cell-progress-line,.cell-progress-fill').forEach(el =>
el.remove());

    const headerTimeLeft = document.getElementById('header-time-left');
    let activePeriodFound = false;

    const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
    if (dayRow) {
        dayRow.querySelector('.day-col').classList.add('is-current-day');
    }

    if (currentDayIndex >= 0 && currentDayIndex <= 4) {
        for (let i = 0; i < timeSlots.length; i++) {

            const startSecs = timeSlots[i].start * 60;
            const endSecs = timeSlots[i].end * 60;

            if (currentTotalSecs >= startSecs && currentTotalSecs < endSecs) {
```

```
            const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);

            if (activeCell) {
                activeCell.classList.add('is-current-class');

                const totalDurationSecs = endSecs - startSecs;
                const elapsedSecs = currentTotalSecs - startSecs;
                const percentDone = (elapsedSecs / totalDurationSecs) * 100;

                // Calculate remaining time and update the header
                const timeRemainingSecs = endSecs - currentTotalSecs;
                const minsLeft = Math.floor(timeRemainingSecs / 60);
                const secsLeft = timeRemainingSecs % 60;

                // We also specify which period is currently running for clarity
                headerTimeLeft.innerText = `${minsLeft}:${secsLeft.toString().padStart(2,
'0')} left in Period ${i + 1}`;
                headerTimeLeft.style.display = "block";
                activePeriodFound = true;

                const line = document.createElement('div');
                line.className = 'cell-progress-line';
                line.style.top = `${percentDone}%`;

                const fill = document.createElement('div');
                fill.className = 'cell-progress-fill';
                fill.style.height = `${percentDone}%`;

                activeCell.appendChild(fill);
                activeCell.appendChild(line);
            }
            break;
        }
    }
}

// If no class is currently happening, hide the header countdown
if (!activePeriodFound) {
    headerTimeLeft.style.display = "none";
}
}
```

```
  init();
</script>

</body>
</html>
```

What I did:

1. **Added a new .time-wrapper group** in the header so the clock and the countdown stay perfectly aligned on the right side.
2. **Removed the tiny .time-left-badge** from CSS and JavaScript completely so your cells only have the red progress bar and the subject name.
3. **Smart Visibility**: The MM:SS left in Period X banner will *only* appear below the main clock if you are actively inside of a time slot. Otherwise, it cleanly hides itself.

can the pade support install app

Yes, we can absolutely do that! By adding two simple files, we can turn your schedule page into a **Progressive Web App (PWA)**.

This means when you open the page on your phone (Safari or Chrome), you will get the option to **"Add to Home Screen"**, and it will look and act exactly like a native app (no browser URL bar, custom icon, opens in full screen). On a computer browser like Chrome, you'll see a little "Install" icon pop up in the address bar.

To make this work, you need to add a **Manifest** and a **Service Worker**.

Here are the 3 quick steps to make it installable. Keep all these files in the exact same folder as your index.html and full_schedule.json.

Step 1: Create the Manifest File (manifest.json)

This file tells the phone/computer what the app's name is, what color the top bar should be, and what icon to use. Create a new file named manifest.json and paste this inside:

```
{
  "name": "My Weekly Schedule",
  "short_name": "Schedule",
  "description": "Live school schedule tracker",
  "start_url": "./index.html",
```

```
  "display": "standalone",
  "background_color": "#f0f2f5",
  "theme_color": "#4a4a4a",
  "icons": [
    {
      "src": "icon.png",
      "sizes": "192x192",
      "type": "image/png"
    }
  ]
}
```

(Note: You will need to save a square image file named icon.png in the same folder to act as your app icon. Any 192x192 or 512x512 square image will work!)

Step 2: Create the Service Worker (sw.js)

This is a background script that allows your app to load even if your phone has no internet connection (offline mode). Create a new file named sw.js and paste this inside:

```
const CACHE_NAME = 'schedule-cache-v1';
const urlsToCache = [
  './index.html',
  './full_schedule.json',
  './manifest.json',
  './icon.png'
];

// Install the service worker and cache the files
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      return cache.addAll(urlsToCache);
    })
  );
});

// Serve cached files when offline
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response => {
```

```
      // Return cached version or fetch from network
      return response || fetch(event.request);
    })
  );
});
```

Step 3: Update your index.html

We need to link the manifest and register the service worker in your main HTML file.

1. **Add this inside your <head> tags** (right below the <title>):

```
<link rel="manifest" href="manifest.json">
<meta name="theme-color" content="#4a4a4a">
<link rel="apple-touch-icon" href="icon.png">
```

2. **Add this inside your <script> tags** (put it right at the top of the <script> section, just above let scheduleData = [];):

```
// Register Service Worker for PWA Installability
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('sw.js').then(registration =>
console.log('ServiceWorker registered')).catch(err => console.log('ServiceWorker
registration failed: ', err));
  });
}
```

Important Rule for Installing (PWAs)

Just like reading the JSON file, **Service Workers and PWAs require a secure connection to work**.

- If you are testing on your computer, your local web server (http://localhost...) will work.
- If you want to install it on your actual phone, you will need to host these files online on a server with an SSL certificate (https://). Free services like **GitHub Pages**, **Vercel**, or **Netlify** are perfect for this and take about 2 minutes to set up.

Would you like me to guide you through hosting this for free on GitHub Pages so you can instantly install it on your phone right now?

still cant se install app

Getting the “Install App” prompt to show up can be surprisingly stubborn because web browsers have very strict security rules for Progressive Web Apps (PWAs).

If you aren’t seeing the install option, it is almost certainly due to one of these three reasons:

1. You are missing the icon.png file

Browsers **will absolutely block** the installation if you do not have an image file named icon.png saved in the exact same folder as your code.

- Find any square image (at least 192x192 pixels).
- Rename it exactly to icon.png.
- Make sure it is in the same folder as index.html.

2. You are opening it as a File (file:///)

If your browser address bar says file:///C:/Users/.../index.html, **it will never work**. PWAs require a secure connection. You *must* run this through a local web server (like the “Live Server” extension in VS Code) so your address bar says http://127.0.0.1:5500 or http://localhost.

3. You need a physical “Install” button on the page

Sometimes the browser hides the install prompt in a menu. Let’s force it out into the open by adding an actual **“Install App” button** right next to your clock!

Here is the code to add a custom install button.

First, add the button to your HTML: Find your section and add the install button so it looks like this:

```
  <div class="controls">
    <button id="install-btn" class="toggle-btn" style="display: none; background: linear-gradient(135deg, #3b82f6 0%, #2563eb 100%);">📲 Install App</button>

    <button id="toggle-view-btn" class="toggle-btn" onclick="toggleViewMode()">Show Today Only</button>
    <div class="time-wrapper">
      <div id="clock" class="clock">00:00:00</div>
```

```
        <div id="header-time-left" class="header-time-left"></div>
    </div>
</div>
```

Next, add the JavaScript to make the button work: Paste this right below your Service Worker registration code at the top of your <script> tag:

```
// --- PWA INSTALLATION LOGIC ---
let deferredPrompt;
const installBtn = document.getElementById('install-btn');

window.addEventListener('beforeinstallprompt', (e) => {
    // Prevent the mini-infobar from appearing on mobile automatically
    e.preventDefault();
    // Stash the event so it can be triggered later.
    deferredPrompt = e;
    // Update UI to show our install button!
    installBtn.style.display = 'block';
});

installBtn.addEventListener('click', async () => {
    if (deferredPrompt) {
        // Show the install prompt
        deferredPrompt.prompt();
        // Wait for the user to respond to the prompt
        const { outcome } = await deferredPrompt.userChoice;
        console.log(`User response to the install prompt: ${outcome}`);
        // We've used the prompt, and can't use it again, throw it away
        deferredPrompt = null;
        // Hide the button after they click it
        installBtn.style.display = 'none';
    }
});

// Hide button if the app is already installed
window.addEventListener('appinstalled', () => {
    installBtn.style.display = 'none';
    console.log('App was successfully installed!');
});
```

How to test this right now:

1. Make sure icon.png, manifest.json, sw.js, full_schedule.json, and index.html are all in the same folder.

2. Start your Local Server (e.g., Live Server in VS Code).
3. Open it in Google Chrome or Microsoft Edge.

If everything is correct, the blue **"📲 Install App"** button will magically appear next to your clock!

Are you running this via Live Server, or are you just double-clicking the HTML file on your computer?

good.after installed success.can i send notificaton when the period start

Yes, you absolutely can! To do this, we need to use the **Web Notifications API**.

There are two important things to know about how web notifications work:

1. **Permission:** You have to ask the user (yourself) for permission first. Browsers block notifications by default.
2. **Mobile Limitations:** On Android, PWA notifications work great. On iPhones (iOS), Apple only recently allowed web notifications, and they only work if the app is actually installed to your Home Screen. Furthermore, for a simple static web app like ours, the app needs to be open (or running in the background) for the script to "tick" and trigger the alert.

I have updated the code to add a **"🔔 Enable Alerts"** button. I also added a memory system so it only sends the notification exactly *once* when the period starts, rather than spamming you every second!

Updated HTML Page (index.html)

Replace your index.html code with this. (Your sw.js, manifest.json, and full_schedule.json stay exactly the same).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Full Weekly Schedule</title>
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#4a4a4a">
  <link rel="apple-touch-icon" href="icon.png">
  <style>
```

```css
:root {
    --header-bg: #4a4a4a;
    --day-bg: #5a5a5a;
    --day-text: #fce4ce;
    --border-color: #222;
    --7a-color: #ffff00;
    --4a-color: #dbeafe;
    --other-color: #ffffff;
    --live-glow: #ff2a2a;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f0f2f5;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 30px 20px;
    margin: 0;
}.header-container {
    width: 100%;
    max-width: 1000px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}.header-title { margin: 0; color: #333; }.controls {
    display: flex;
    gap: 15px;
    align-items: flex-start;
    flex-wrap: wrap;
    justify-content: flex-end;
}.toggle-btn {
    padding: 10px 16px;
    margin-top: 2px;
    background: linear-gradient(135deg, #10b981 0%, #059669 100%);
    color: white;
    border: none;
    border-radius: 8px;
    font-weight: bold;
    font-size: 0.95rem;
    cursor: pointer;
    box-shadow: 0 4px 6px rgba(0,0,0,0.2), inset 0 2px 0 rgba(255,255,255,0.2);
```

```
    transition: all 0.2s;
}.toggle-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 6px 8px rgba(0,0,0,0.3);
}.time-wrapper {
    display: flex;
    flex-direction: column;
    align-items: flex-end;
    gap: 6px;
}.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}.header-time-left {
    font-size: 0.85rem;
    font-weight: bold;
    color: var(--live-glow);
    background: #ffebe9;
    padding: 4px 12px;
    border-radius: 6px;
    border: 1px solid rgba(255, 42, 42, 0.2);
    display: none;
    animation: pulse 2s infinite;
}

@keyframes pulse {
    0% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0.4); }
    70% { box-shadow: 0 0 0 6px rgba(255, 42, 42, 0); }
    100% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0); }
}

table {
    width: 100%;
    max-width: 1000px;
    border-collapse: collapse;
    background: white;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    border: 2px solid var(--border-color);
    transition: all 0.3s ease;
```

```
    }

    th, td {
      border: 1px solid var(--border-color);
      text-align: center;
      padding: 12px 5px;
      width: 11%;
    }

    th {
      background-color: var(--header-bg);
      color: white;
      font-size: 0.9rem;
      line-height: 1.4;
    }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }.day-col {
      background-color: var(--day-bg);
      color: var(--day-text);
      font-size: 1.1rem;
      font-weight: bold;
      text-align: left;
      padding-left: 15px;
      width: 12%;
    }.subject-cell {
      font-size: 1.2rem;
      font-weight: 900;
      color: #111;
      position: relative;
      transition: all 0.3s;
      overflow: hidden;
    }.nafs-text {
      display: block;
      font-size: 0.7rem;
      font-weight: normal;
      color: #444;
      margin-top: 5px;
      position: relative;
      z-index: 5;
    }.subject-cell span.main-text {
      position: relative;
      z-index: 5;
    }.sub-7A { background-color: var(--7a-color); }.sub-4A { background-color: var(--4a-
color); }.sub-other { background-color: var(--other-color); }.sub-free { background-
```

```
color: white; }.is-current-class {
        box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4);
        z-index: 10;
    }.is-current-day {
        border-left: 5px solid var(--live-glow);
    }.cell-progress-line {
        position: absolute;
        left: 0;
        right: 0;
        height: 3px;
        background: rgba(255, 42, 42, 0.8);
        z-index: 1;
        pointer-events: none;
        box-shadow: 0 0 8px rgba(255, 42, 42, 0.6);
    }.cell-progress-line::before {
        content: '';
        position: absolute;
        left: 0;
        top: -4px;
        width: 10px;
        height: 10px;
        background: var(--live-glow);
        border-radius: 50%;
        box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
    }.cell-progress-fill {
        position: absolute;
        top: 0;
        left: 0;
        right: 0;
        background: rgba(255, 42, 42, 0.1);
        z-index: 0;
        pointer-events: none;
    }.error-message {
        color: red;
        font-weight: bold;
        padding: 20px;
        background: #ffebe9;
        border: 1px solid red;
        border-radius: 8px;
        margin-top: 20px;
        display: none;
    }
</style>
```

```
</head>
<body>

<div class="header-container">
  <h2 class="header-title" id="main-title">Full Weekly View</h2>
  <div class="controls">
    <button id="notify-btn" class="toggle-btn" onclick="requestNotificationPermission()" style="display: none; background: linear-gradient(135deg, #8b5cf6 0%, #6d28d9 100%);">🔔 Enable Alerts</button>

    <button id="install-btn" class="toggle-btn" style="display: none; background: linear-gradient(135deg, #3b82f6 0%, #2563eb 100%);">📲 Install App</button>

    <button id="toggle-view-btn" class="toggle-btn" onclick="toggleViewMode()">Show Today Only</button>

    <div class="time-wrapper">
      <div id="clock" class="clock">00:00:00</div>
      <div id="header-time-left" class="header-time-left"></div>
    </div>
  </div>
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
  <thead>
    <tr>
      <th>Day</th>
      <th><span class="period-num">1</span><span class="period-time">(9:00 - 9:30)</span></th>
      <th><span class="period-num">2</span><span class="period-time">(9:30 - 10:00)</span></th>
      <th><span class="period-num">3</span><span class="period-time">(10:00 - 10:30)</span></th>
      <th><span class="period-num">4</span><span class="period-time">(10:30 - 11:00)</span></th>
      <th><span class="period-num">5</span><span class="period-time">(11:00 - 11:30)</span></th>
      <th><span class="period-num">6</span><span class="period-time">(11:30 - 12:00)</span></th>
      <th><span class="period-num">7</span><span class="period-time">(12:00 -
```

```
12:25)</span></th>
            <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
        </tr>
    </thead>
    <tbody id="table-body">
    </tbody>
</table>

<script>
    let scheduleData = [];
    let isShowingAllDays = true;
    let currentActivePeriodKey = null; // Memory to prevent spamming notifications

    const timeSlots = [
        { start: 9*60, end: 9*60+30 },
        { start: 9*60+30, end: 10*60 },
        { start: 10*60, end: 10*60+30 },
        { start: 10*60+30, end: 11*60 },
        { start: 11*60, end: 11*60+30 },
        { start: 11*60+30, end: 12*60 },
        { start: 12*60, end: 12*60+25 },
        { start: 12*60+25, end: 12*60+50 }
    ];

    // --- PWA SERVICE WORKER & INSTALL LOGIC ---
    if ('serviceWorker' in navigator) {
        window.addEventListener('load', () => {
            navigator.serviceWorker.register('sw.js');
        });
    }

    let deferredPrompt;
    const installBtn = document.getElementById('install-btn');

    window.addEventListener('beforeinstallprompt', (e) => {
        e.preventDefault();
        deferredPrompt = e;
        installBtn.style.display = 'block';
    });

    installBtn.addEventListener('click', async () => {
```

```
    if (deferredPrompt) {
        deferredPrompt.prompt();
        const { outcome } = await deferredPrompt.userChoice;
        deferredPrompt = null;
        installBtn.style.display = 'none';
    }
});

window.addEventListener('appinstalled', () => {
    installBtn.style.display = 'none';
});


// --- NOTIFICATION LOGIC ---
function checkNotificationStatus() {
    if ("Notification" in window) {
        if (Notification.permission === "default") {
            document.getElementById('notify-btn').style.display = 'block';
        }
    }
}

function requestNotificationPermission() {
    if ("Notification" in window) {
        Notification.requestPermission().then(permission => {
            if (permission === "granted") {
                document.getElementById('notify-btn').style.display = 'none';
                sendNotification("Alerts Enabled", "You will be notified when classes start!");
            }
        });
    }
}

function sendNotification(title, bodyText) {
    if ("Notification" in window && Notification.permission === "granted") {
        navigator.serviceWorker.ready.then(registration => {
            registration.showNotification(title, {
                body: bodyText,
                icon: 'icon.png',
                vibrate: [200, 100, 200] // Makes phones vibrate!
            });
        });
```

```
    }
  }


  // --- DATA & UI LOGIC ---
  async function init() {
    checkNotificationStatus();

    try {
      const response = await fetch('full_schedule.json');
      if (!response.ok) throw new Error("HTTP error " + response.status);

      scheduleData = await response.json();

      buildTable();
      updateLiveIndicator();
      setInterval(updateLiveIndicator, 1000);

    } catch (error) {
      console.error("Failed to load:", error);
      const errorBox = document.getElementById('error-box');
      errorBox.style.display = "block";
      errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you
are using a local web server.`;
      document.getElementById('schedule-table').style.display = "none";
    }
  }

  function buildTable() {
    const tbody = document.getElementById('table-body');
    tbody.innerHTML = '';

    scheduleData.forEach(dayRow => {
      const tr = document.createElement('tr');
      tr.id = `row-day-${dayRow.dayIndex}`;

      let html = `<td class="day-col">${dayRow.day}</td>`;

      dayRow.classes.forEach((c, index) => {
        let colorClass = 'sub-free';
        if (c.sub === '7A') colorClass = 'sub-7A';
        else if (c.sub === '4A') colorClass = 'sub-4A';
```

```
            else if (c.sub!== '') colorClass = 'sub-other';

            const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

            html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-${index}">
                        <span class="main-text">${c.sub}</span>
                        ${nafsHTML}
                    </td>`;
        });

        tr.innerHTML = html;
        tbody.appendChild(tr);
    });

    applyViewFilter();
}

function toggleViewMode() {
    isShowingAllDays =!isShowingAllDays;
    const btn = document.getElementById('toggle-view-btn');
    const title = document.getElementById('main-title');

    if (isShowingAllDays) {
        btn.innerText = "Show Today Only";
        title.innerText = "Full Weekly View";
    } else {
        btn.innerText = "Show All Days";
        title.innerText = "Today's View";
    }
    applyViewFilter();
}

function applyViewFilter() {
    let currentDayIndex = new Date().getDay();
    if (currentDayIndex > 4) currentDayIndex = 0;

    scheduleData.forEach(dayRow => {
        const tr = document.getElementById(`row-day-${dayRow.dayIndex}`);
        if (!tr) return;
        tr.style.display = (isShowingAllDays || dayRow.dayIndex === currentDayIndex)? '' : 'none';
```

```
    });
  }

  function updateLiveIndicator() {
    if (scheduleData.length === 0) return;

    const now = new Date();
    const currentDayIndex = now.getDay();
    const currentTotalSecs = now.getHours() * 3600 + now.getMinutes() * 60 +
now.getSeconds();

    document.getElementById('clock').innerText =
      now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' });

    document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-
current-class'));
    document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-
current-day'));
    document.querySelectorAll('.cell-progress-line,.cell-progress-fill').forEach(el =>
el.remove());

    const headerTimeLeft = document.getElementById('header-time-left');
    let activePeriodFound = false;

    const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
    if (dayRow) dayRow.querySelector('.day-col').classList.add('is-current-day');

    if (currentDayIndex >= 0 && currentDayIndex <= 4) {
      for (let i = 0; i < timeSlots.length; i++) {

        const startSecs = timeSlots[i].start * 60;
        const endSecs = timeSlots[i].end * 60;

        if (currentTotalSecs >= startSecs && currentTotalSecs < endSecs) {
          const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);

          if (activeCell) {
            activeCell.classList.add('is-current-class');
            activePeriodFound = true;

            // --- NEW: Trigger Notification if it's a new period ---
            const periodKey = `${currentDayIndex}-${i}`;
```

```
            if (currentActivePeriodKey !== periodKey) {
                currentActivePeriodKey = periodKey; // Save to memory so it doesn't alert again

                // Find out what subject it is
                const subjectText = activeCell.querySelector('.main-text').innerText;
                const isFree = subjectText.trim() === "";

                if (!isFree) {
                    sendNotification(`Period ${i + 1} Started!`, `Time for ${subjectText}`);
                } else {
                    sendNotification(`Period ${i + 1} Started`, `You have a free period.`);
                }
            }

            // Update Visuals
            const totalDurationSecs = endSecs - startSecs;
            const elapsedSecs = currentTotalSecs - startSecs;
            const percentDone = (elapsedSecs / totalDurationSecs) * 100;

            const timeRemainingSecs = endSecs - currentTotalSecs;
            const minsLeft = Math.floor(timeRemainingSecs / 60);
            const secsLeft = timeRemainingSecs % 60;

            headerTimeLeft.innerText = `${minsLeft}:${secsLeft.toString().padStart(2, '0')} left in Period ${i + 1}`;
            headerTimeLeft.style.display = "block";

            const line = document.createElement('div');
            line.className = 'cell-progress-line';
            line.style.top = `${percentDone}%`;

            const fill = document.createElement('div');
            fill.className = 'cell-progress-fill';
            fill.style.height = `${percentDone}%`;

            activeCell.appendChild(fill);
            activeCell.appendChild(line);
        }
        break;
    }
}
```

```
    }

    // If no class is running, reset memory and hide header
    if (!activePeriodFound) {
      headerTimeLeft.style.display = "none";
      currentActivePeriodKey = null;
    }
  }

  init();
</script>

</body>
</html>
```

How to use this:

1. When you open the page, you will see a purple **"🔔 Enable Alerts"** button.
2. Click it, and your browser will ask, *"Allow notifications?"* Click **Allow**.
3. Now, whenever a new time slot hits (e.g., exactly at 9:30 AM), your phone/computer will pop up a notification saying: **"Period 2 Started - You have a free period."** or **"Period 3 Started - Time for 5A"**.

Remember, for standard mobile web browsers, keep the tab open (or running in the background) for the script to accurately check the time and send the alert.

can the page play this sound when start period and notification.

Yes, we can absolutely make the page play that notification sound!

To do this, we will use JavaScript's built-in Audio object to trigger the sound at the exact same time the visual notification pops up.

Here is the updated index.html code.

What you need to do first:

1. Download the audio file you just provided.
2. Save it in the **exact same folder** as your index.html and name it exactly **notification1.mp3**.

Updated HTML Page (index.html)

Replace your index.html code with this:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Full Weekly Schedule</title>
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#4a4a4a">
  <link rel="apple-touch-icon" href="icon.png">
  <style>
    :root {
      --header-bg: #4a4a4a;
      --day-bg: #5a5a5a;
      --day-text: #fce4ce;
      --border-color: #222;
      --7a-color: #ffff00;
      --4a-color: #dbeafe;
      --other-color: #ffffff;
      --live-glow: #ff2a2a;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: #f0f2f5;
      display: flex;
      flex-direction: column;
      align-items: center;
      padding: 30px 20px;
      margin: 0;
    }.header-container {
      width: 100%;
      max-width: 1000px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      margin-bottom: 20px;
    }.header-title { margin: 0; color: #333; }.controls {
      display: flex;
      gap: 15px;
```

```
    align-items: flex-start;
    flex-wrap: wrap;
    justify-content: flex-end;
}.toggle-btn {
    padding: 10px 16px;
    margin-top: 2px;
    background: linear-gradient(135deg, #10b981 0%, #059669 100%);
    color: white;
    border: none;
    border-radius: 8px;
    font-weight: bold;
    font-size: 0.95rem;
    cursor: pointer;
    box-shadow: 0 4px 6px rgba(0,0,0,0.2), inset 0 2px 0 rgba(255,255,255,0.2);
    transition: all 0.2s;
}.toggle-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 6px 8px rgba(0,0,0,0.3);
}.time-wrapper {
    display: flex;
    flex-direction: column;
    align-items: flex-end;
    gap: 6px;
}.clock {
    font-size: 1.2rem;
    font-weight: bold;
    color: #3b82f6;
    background: white;
    padding: 8px 16px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}.header-time-left {
    font-size: 0.85rem;
    font-weight: bold;
    color: var(--live-glow);
    background: #ffebe9;
    padding: 4px 12px;
    border-radius: 6px;
    border: 1px solid rgba(255, 42, 42, 0.2);
    display: none;
    animation: pulse 2s infinite;
}
```

```
        @keyframes pulse {
            0% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0.4); }
            70% { box-shadow: 0 0 0 6px rgba(255, 42, 42, 0); }
            100% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0); }
        }

        table {
            width: 100%;
            max-width: 1000px;
            border-collapse: collapse;
            background: white;
            box-shadow: 0 10px 30px rgba(0,0,0,0.1);
            border: 2px solid var(--border-color);
            transition: all 0.3s ease;
        }

        th, td {
            border: 1px solid var(--border-color);
            text-align: center;
            padding: 12px 5px;
            width: 11%;
        }

        th {
            background-color: var(--header-bg);
            color: white;
            font-size: 0.9rem;
            line-height: 1.4;
        }.period-num { font-size: 1.1rem; font-weight: bold; display: block; }.period-time {
font-size: 0.75rem; color: #ccc; }.day-col {
            background-color: var(--day-bg);
            color: var(--day-text);
            font-size: 1.1rem;
            font-weight: bold;
            text-align: left;
            padding-left: 15px;
            width: 12%;
        }.subject-cell {
            font-size: 1.2rem;
            font-weight: 900;
            color: #111;
            position: relative;
```

```
        transition: all 0.3s;
        overflow: hidden;
    }.nafs-text {
        display: block;
        font-size: 0.7rem;
        font-weight: normal;
        color: #444;
        margin-top: 5px;
        position: relative;
        z-index: 5;
    }.subject-cell span.main-text {
        position: relative;
        z-index: 5;
    }.sub-7A { background-color: var(--7a-color); }.sub-4A { background-color: var(--4a-color); }.sub-other { background-color: var(--other-color); }.sub-free { background-color: white; }.is-current-class {
        box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4);
        z-index: 10;
    }.is-current-day {
        border-left: 5px solid var(--live-glow);
    }.cell-progress-line {
        position: absolute;
        left: 0;
        right: 0;
        height: 3px;
        background: rgba(255, 42, 42, 0.8);
        z-index: 1;
        pointer-events: none;
        box-shadow: 0 0 8px rgba(255, 42, 42, 0.6);
    }.cell-progress-line::before {
        content: '';
        position: absolute;
        left: 0;
        top: -4px;
        width: 10px;
        height: 10px;
        background: var(--live-glow);
        border-radius: 50%;
        box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
    }.cell-progress-fill {
        position: absolute;
        top: 0;
        left: 0;
```

```
            right: 0;
            background: rgba(255, 42, 42, 0.1);
            z-index: 0;
            pointer-events: none;
        }.error-message {
            color: red;
            font-weight: bold;
            padding: 20px;
            background: #ffebe9;
            border: 1px solid red;
            border-radius: 8px;
            margin-top: 20px;
            display: none;
        }
    </style>
</head>
<body>

<div class="header-container">
    <h2 class="header-title" id="main-title">Full Weekly View</h2>
    <div class="controls">
        <button id="notify-btn" class="toggle-btn" onclick="requestNotificationPermission()" style="display: none; background: linear-gradient(135deg, #8b5cf6 0%, #6d28d9 100%);">🔔 Enable Alerts</button>
        <button id="install-btn" class="toggle-btn" style="display: none; background: linear-gradient(135deg, #3b82f6 0%, #2563eb 100%);">📲 Install App</button>
        <button id="toggle-view-btn" class="toggle-btn" onclick="toggleViewMode()">Show Today Only</button>

        <div class="time-wrapper">
            <div id="clock" class="clock">00:00:00</div>
            <div id="header-time-left" class="header-time-left"></div>
        </div>
    </div>
</div>

<div id="error-box" class="error-message"></div>

<table id="schedule-table">
    <thead>
        <tr>
            <th>Day</th>
```

```
        <th><span class="period-num">1</span><span class="period-time">(9:00 -
9:30)</span></th>
        <th><span class="period-num">2</span><span class="period-time">(9:30 -
10:00)</span></th>
        <th><span class="period-num">3</span><span class="period-time">(10:00 -
10:30)</span></th>
        <th><span class="period-num">4</span><span class="period-time">(10:30 -
11:00)</span></th>
        <th><span class="period-num">5</span><span class="period-time">(11:00 -
11:30)</span></th>
        <th><span class="period-num">6</span><span class="period-time">(11:30 -
12:00)</span></th>
        <th><span class="period-num">7</span><span class="period-time">(12:00 -
12:25)</span></th>
        <th><span class="period-num">8</span><span class="period-time">(12:25 -
12:50)</span></th>
      </tr>
    </thead>
    <tbody id="table-body">
    </tbody>
</table>

<script>
    let scheduleData = [];
    let isShowingAllDays = true;
    let currentActivePeriodKey = null;

    // Load the audio file
    const alertSound = new Audio('notification1.mp3');

    const timeSlots = [
        { start: 9*60, end: 9*60+30 },
        { start: 9*60+30, end: 10*60 },
        { start: 10*60, end: 10*60+30 },
        { start: 10*60+30, end: 11*60 },
        { start: 11*60, end: 11*60+30 },
        { start: 11*60+30, end: 12*60 },
        { start: 12*60, end: 12*60+25 },
        { start: 12*60+25, end: 12*60+50 }
    ];

    // --- PWA SERVICE WORKER & INSTALL LOGIC ---
```

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('sw.js');
  });
}

let deferredPrompt;
const installBtn = document.getElementById('install-btn');

window.addEventListener('beforeinstallprompt', (e) => {
  e.preventDefault();
  deferredPrompt = e;
  installBtn.style.display = 'block';
});

installBtn.addEventListener('click', async () => {
  if (deferredPrompt) {
    deferredPrompt.prompt();
    const { outcome } = await deferredPrompt.userChoice;
    deferredPrompt = null;
    installBtn.style.display = 'none';
  }
});

window.addEventListener('appinstalled', () => {
  installBtn.style.display = 'none';
});



// --- NOTIFICATION & AUDIO LOGIC ---
function checkNotificationStatus() {
  if ("Notification" in window) {
    if (Notification.permission === "default") {
      document.getElementById('notify-btn').style.display = 'block';
    }
  }
}

function requestNotificationPermission() {
  if ("Notification" in window) {
    Notification.requestPermission().then(permission => {
      if (permission === "granted") {
```

```
                document.getElementById('notify-btn').style.display = 'none';
                // Play a quick test sound to ensure browser allows audio playback on this page
                alertSound.play().catch(e => console.log("Test audio play skipped."));
                sendNotification("Alerts Enabled", "You will be notified when classes start!");
            }
        });
    }
}

function sendNotification(title, bodyText) {
    if ("Notification" in window && Notification.permission === "granted") {
        navigator.serviceWorker.ready.then(registration => {
            registration.showNotification(title, {
                body: bodyText,
                icon: 'icon.png',
                vibrate: [200, 100, 200]
            });
        });
    }
}


// --- DATA & UI LOGIC ---
async function init() {
    checkNotificationStatus();

    try {
        const response = await fetch('full_schedule.json');
        if (!response.ok) throw new Error("HTTP error " + response.status);

        scheduleData = await response.json();

        buildTable();
        updateLiveIndicator();
        setInterval(updateLiveIndicator, 1000);

    } catch (error) {
        console.error("Failed to load:", error);
        const errorBox = document.getElementById('error-box');
        errorBox.style.display = "block";
        errorBox.innerHTML = `Could not load full_schedule.json.<br><br>Make sure you are using a local web server.`;
```

```
        document.getElementById('schedule-table').style.display = "none";
    }
}

function buildTable() {
    const tbody = document.getElementById('table-body');
    tbody.innerHTML = '';

    scheduleData.forEach(dayRow => {
        const tr = document.createElement('tr');
        tr.id = `row-day-${dayRow.dayIndex}`;

        let html = `<td class="day-col">${dayRow.day}</td>`;

        dayRow.classes.forEach((c, index) => {
            let colorClass = 'sub-free';
            if (c.sub === '7A') colorClass = 'sub-7A';
            else if (c.sub === '4A') colorClass = 'sub-4A';
            else if (c.sub!== '') colorClass = 'sub-other';

            const nafsHTML = c.nafs? `<span class="nafs-text">(NAFS M)</span>` : '';

            html += `<td class="subject-cell ${colorClass}" id="cell-${dayRow.dayIndex}-${index}">
                        <span class="main-text">${c.sub}</span>
                        ${nafsHTML}
                    </td>`;
        });

        tr.innerHTML = html;
        tbody.appendChild(tr);
    });

    applyViewFilter();
}

function toggleViewMode() {
    isShowingAllDays =!isShowingAllDays;
    const btn = document.getElementById('toggle-view-btn');
    const title = document.getElementById('main-title');

    if (isShowingAllDays) {
```

```
            btn.innerText = "Show Today Only";
            title.innerText = "Full Weekly View";
        } else {
            btn.innerText = "Show All Days";
            title.innerText = "Today's View";
        }
        applyViewFilter();
    }

    function applyViewFilter() {
        let currentDayIndex = new Date().getDay();
        if (currentDayIndex > 4) currentDayIndex = 0;

        scheduleData.forEach(dayRow => {
            const tr = document.getElementById(`row-day-${dayRow.dayIndex}`);
            if (!tr) return;
            tr.style.display = (isShowingAllDays || dayRow.dayIndex === currentDayIndex)? ''
: 'none';
        });
    }

    function updateLiveIndicator() {
        if (scheduleData.length === 0) return;

        const now = new Date();
        const currentDayIndex = now.getDay();
        const currentTotalSecs = now.getHours() * 3600 + now.getMinutes() * 60 +
now.getSeconds();

        document.getElementById('clock').innerText =
            now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' });

        document.querySelectorAll('.is-current-class').forEach(el => el.classList.remove('is-
current-class'));
        document.querySelectorAll('.is-current-day').forEach(el => el.classList.remove('is-
current-day'));
        document.querySelectorAll('.cell-progress-line,.cell-progress-fill').forEach(el =>
el.remove());

        const headerTimeLeft = document.getElementById('header-time-left');
        let activePeriodFound = false;
```

```
        const dayRow = document.getElementById(`row-day-${currentDayIndex}`);
        if (dayRow) dayRow.querySelector('.day-col').classList.add('is-current-day');

        if (currentDayIndex >= 0 && currentDayIndex <= 4) {
            for (let i = 0; i < timeSlots.length; i++) {

                const startSecs = timeSlots[i].start * 60;
                const endSecs = timeSlots[i].end * 60;

                if (currentTotalSecs >= startSecs && currentTotalSecs < endSecs) {
                    const activeCell = document.getElementById(`cell-${currentDayIndex}-${i}`);

                    if (activeCell) {
                        activeCell.classList.add('is-current-class');
                        activePeriodFound = true;

                        // --- TRIGGER NOTIFICATION & PLAY AUDIO ---
                        const periodKey = `${currentDayIndex}-${i}`;
                        if (currentActivePeriodKey!== periodKey) {
                            currentActivePeriodKey = periodKey;

                            // Play the custom audio file
                            alertSound.play().catch(e => console.log("Audio blocked by browser
autoplay rules. Please interact with the page first."));

                            const subjectText = activeCell.querySelector('.main-text').innerText;
                            const isFree = subjectText.trim() === "";

                            if (!isFree) {
                                sendNotification(`Period ${i + 1} Started!`, `Time for ${subjectText}`);
                            } else {
                                sendNotification(`Period ${i + 1} Started`, `You have a free period.`);
                            }
                        }

                        // Update Visuals
                        const totalDurationSecs = endSecs - startSecs;
                        const elapsedSecs = currentTotalSecs - startSecs;
                        const percentDone = (elapsedSecs / totalDurationSecs) * 100;

                        const timeRemainingSecs = endSecs - currentTotalSecs;
                        const minsLeft = Math.floor(timeRemainingSecs / 60);
```

```
                const secsLeft = timeRemainingSecs % 60;

                headerTimeLeft.innerText = `${minsLeft}:${secsLeft.toString().padStart(2,
'0')} left in Period ${i + 1}`;
                headerTimeLeft.style.display = "block";

                const line = document.createElement('div');
                line.className = 'cell-progress-line';
                line.style.top = `${percentDone}%`;

                const fill = document.createElement('div');
                fill.className = 'cell-progress-fill';
                fill.style.height = `${percentDone}%`;

                activeCell.appendChild(fill);
                activeCell.appendChild(line);
              }
              break;
            }
          }
        }

        if (!activePeriodFound) {
          headerTimeLeft.style.display = "none";
          currentActivePeriodKey = null;
        }
      }

      init();
</script>

</body>
</html>
```

⚠️ Important Note About Browser Sound Rules (Auto-play Policy):

Modern browsers (like Chrome, Safari, and Edge) will **block** websites from playing sounds on their own unless you have interacted with the page first (by clicking a button or tapping the screen).

To ensure the sound isn't blocked when the period starts:

- Make sure you click **“Show Today Only”** or **“Enable Alerts”** at least once when you open the app. That initial click proves to the browser that you are actively using the page, and it will allow the notification1.mp3 file to play automatically later!

to vue js component.separate code to reusable compoents easy to maintain and improve

Transitioning to **Vue.js** is a fantastic decision. It will make your code much cleaner, easier to maintain, and simple to expand in the future.

To achieve this, we will break your app down into a **Component Tree** using modern Vue 3 (<script setup> syntax):

1. **App.vue**: The main parent. It fetches the JSON, manages the live timer, handles notifications, and passes data down.
2. **AppHeader.vue**: Contains the title, clock, time-left badge, and action buttons.
3. **ScheduleGrid.vue**: The table structure (headers and rows).
4. **ScheduleCell.vue**: A reusable component for a single class period (handles the red progress bar and colors).

Here is how to structure your Vue application.

1. The Main App (src/App.vue)

This acts as the “brain” of your app. It handles the state (time, data, PWA, notifications) and passes the relevant pieces down to the child components.

```
<template>
  <div class="app-container">
    <AppHeader
      :is-showing-all-days="isShowingAllDays"
      :current-time-string="currentTimeString"
      :active-period="activePeriodDetails"
      :can-install="canInstall"
      :notifications-enabled="notificationsEnabled"
      @toggle-view="isShowingAllDays =!isShowingAllDays"
      @install-app="installPWA"
      @enable-notifications="requestNotifications"
    />
```

```
    <div v-if="errorMessage" class="error-message" v-html="errorMessage"></div>

    <ScheduleGrid
      v-else
      :schedule-data="scheduleData"
      :is-showing-all-days="isShowingAllDays"
      :current-day-index="currentDayIndex"
      :active-period="activePeriodDetails"
    />
  </div>
</template>

<script setup>
import { ref, computed, onMounted, watch } from 'vue';
import AppHeader from './components/AppHeader.vue';
import ScheduleGrid from './components/ScheduleGrid.vue';

// State
const scheduleData = ref([]);
const isShowingAllDays = ref(true);
const currentTimeSecs = ref(0);
const currentTimeString = ref('00:00:00');
const currentDayIndex = ref(0);
const errorMessage = ref('');

// PWA & Notifications State
const canInstall = ref(false);
let deferredPrompt = null;
const notificationsEnabled = ref(Notification.permission === 'granted');
const alertSound = new Audio('/notification1.mp3');

// Constants
const timeSlots = [
  { start: 9*60, end: 9*60+30 }, { start: 9*60+30, end: 10*60 },
  { start: 10*60, end: 10*60+30 }, { start: 10*60+30, end: 11*60 },
  { start: 11*60, end: 11*60+30 }, { start: 11*60+30, end: 12*60 },
  { start: 12*60, end: 12*60+25 }, { start: 12*60+25, end: 12*60+50 }
];

// Computed: Master logic for active class tracking
const activePeriodDetails = computed(() => {
  if (currentDayIndex.value > 4) return null; // Weekend
```

```
  for (let i = 0; i < timeSlots.length; i++) {
    const startSecs = timeSlots[i].start * 60;
    const endSecs = timeSlots[i].end * 60;

    if (currentTimeSecs.value >= startSecs && currentTimeSecs.value < endSecs) {
      const totalDuration = endSecs - startSecs;
      const elapsed = currentTimeSecs.value - startSecs;
      const rem = endSecs - currentTimeSecs.value;

      const dayData = scheduleData.value.find(d => d.dayIndex ===
currentDayIndex.value);
      const subject = dayData? dayData.classes[i].sub : '';

      return {
        dayIndex: currentDayIndex.value,
        periodIndex: i,
        progressPercent: (elapsed / totalDuration) * 100,
        timeLeftStr: `${Math.floor(rem / 60)}:${(rem % 60).toString().padStart(2, '0')} left in
Period ${i + 1}`,
        subject: subject
      };
    }
  }
  return null;
});

// Watcher: Triggers Audio & Web Notification when a NEW period starts
watch(() => activePeriodDetails.value?.periodIndex, (newVal, oldVal) => {
  if (newVal!== undefined && newVal!== oldVal && activePeriodDetails.value) {
    alertSound.play().catch(() => console.log("Audio blocked by browser."));
    const sub = activePeriodDetails.value.subject;
    const body = sub? `Time for ${sub}` : `You have a free period.`;

    if (notificationsEnabled.value) {
      navigator.serviceWorker.ready.then(reg => {
        reg.showNotification(`Period ${newVal + 1} Started!`, { body, icon: '/icon.png',
vibrate: [200, 100, 200] });
      });
    }
  }
});
```

```
// Setup Timer
const updateClock = () => {
  const now = new Date();
  currentDayIndex.value = now.getDay();
  currentTimeSecs.value = now.getHours() * 3600 + now.getMinutes() * 60 +
now.getSeconds();
  currentTimeString.value = now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit',
second: '2-digit' });
};

// Actions
const requestNotifications = () => {
  Notification.requestPermission().then(perm => {
    if (perm === "granted") {
      notificationsEnabled.value = true;
      alertSound.play().catch(() => {}); // Unlock audio context
    }
  });
};

const installPWA = async () => {
  if (!deferredPrompt) return;
  deferredPrompt.prompt();
  await deferredPrompt.userChoice;
  deferredPrompt = null;
  canInstall.value = false;
};

// Lifecycle Hooks
onMounted(async () => {
  updateClock();
  setInterval(updateClock, 1000);

  // Fetch Schedule Data
  try {
    const res = await fetch('/full_schedule.json');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    scheduleData.value = await res.json();
  } catch (e) {
    errorMessage.value = `Failed to load schedule.<br>${e.message}`;
  }
```

```
  // PWA Setup
  window.addEventListener('beforeinstallprompt', (e) => {
    e.preventDefault();
    deferredPrompt = e;
    canInstall.value = true;
  });
  window.addEventListener('appinstalled', () => canInstall.value = false);
});
</script>

<style>
/* Global CSS variables */
:root {
  --header-bg: #4a4a4a;
  --day-bg: #5a5a5a;
  --day-text: #fce4ce;
  --border-color: #222;
  --7a-color: #ffff00;
  --4a-color: #dbeafe;
  --other-color: #ffffff;
  --live-glow: #ff2a2a;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f0f2f5;
  margin: 0;
  display: flex;
  justify-content: center;
}.app-container {
  width: 100%;
  max-width: 1000px;
  padding: 30px 20px;
  display: flex;
  flex-direction: column;
}.error-message {
  color: red;
  font-weight: bold;
  padding: 20px;
  background: #ffebe9;
  border: 1px solid red;
```

```
  border-radius: 8px;
}
</style>
```

---

2. Header Component (src/components/AppHeader.vue)

This isolates all the controls at the top of your page. It takes props from App.vue and $emits events when buttons are clicked.

```
<template>
  <div class="header-container">
    <h2 class="header-title">{{ isShowingAllDays? 'Full Weekly View' : 'Today\'s View' }}</h2>

    <div class="controls">
      <button v-if="!notificationsEnabled" class="toggle-btn btn-purple" @click="$emit('enable-notifications')">
        🔔 Enable Alerts
      </button>

      <button v-if="canInstall" class="toggle-btn btn-blue" @click="$emit('install-app')">
        📲 Install App
      </button>

      <button class="toggle-btn btn-green" @click="$emit('toggle-view')">
        {{ isShowingAllDays? 'Show Today Only' : 'Show All Days' }}
      </button>

      <div class="time-wrapper">
        <div class="clock">{{ currentTimeString }}</div>
        <div v-if="activePeriod" class="header-time-left">
          {{ activePeriod.timeLeftStr }}
        </div>
      </div>
    </div>
  </div>
</template>

<script setup>
defineProps({
  isShowingAllDays: Boolean,
  currentTimeString: String,
```

```
  activePeriod: Object,
  canInstall: Boolean,
  notificationsEnabled: Boolean
});
defineEmits(['toggle-view', 'install-app', 'enable-notifications']);
</script>

<style scoped>.header-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}.header-title { margin: 0; color: #333; }.controls {
  display: flex;
  gap: 15px;
  align-items: flex-start;
  flex-wrap: wrap;
  justify-content: flex-end;
}.toggle-btn {
  padding: 10px 16px; margin-top: 2px;
  color: white; border: none; border-radius: 8px;
  font-weight: bold; font-size: 0.95rem; cursor: pointer;
  box-shadow: 0 4px 6px rgba(0,0,0,0.2), inset 0 2px 0 rgba(255,255,255,0.2);
  transition: all 0.2s;
}.toggle-btn:hover { transform: translateY(-1px); box-shadow: 0 6px 8px
rgba(0,0,0,0.3); }.btn-green { background: linear-gradient(135deg, #10b981 0%, #059669
100%); }.btn-blue { background: linear-gradient(135deg, #3b82f6 0%, #2563eb
100%); }.btn-purple { background: linear-gradient(135deg, #8b5cf6 0%, #6d28d9
100%); }.time-wrapper { display: flex; flex-direction: column; align-items: flex-end; gap:
6px; }.clock {
  font-size: 1.2rem; font-weight: bold; color: #3b82f6;
  background: white; padding: 8px 16px; border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}.header-time-left {
  font-size: 0.85rem; font-weight: bold; color: var(--live-glow);
  background: #ffebe9; padding: 4px 12px; border-radius: 6px;
  border: 1px solid rgba(255, 42, 42, 0.2);
  animation: pulse 2s infinite;
}
@keyframes pulse {
  0% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0.4); }
  70% { box-shadow: 0 0 0 6px rgba(255, 42, 42, 0); }
  100% { box-shadow: 0 0 0 0 rgba(255, 42, 42, 0); }
```

```
}
</style>
```

---

3. Schedule Grid (src/components/ScheduleGrid.vue)

This component renders the table structure and uses v-for to generate rows. Notice how much smaller and easier to read this is compared to the plain HTML version!

```
<template>
  <table class="schedule-table">
    <thead>
      <tr>
        <th>Day</th>
        <th><span class="p-num">1</span><span class="p-time">(9:00 - 9:30)</span></th>
        <th><span class="p-num">2</span><span class="p-time">(9:30 - 10:00)</span></th>
        <th><span class="p-num">3</span><span class="p-time">(10:00 - 10:30)</span></th>
        <th><span class="p-num">4</span><span class="p-time">(10:30 - 11:00)</span></th>
        <th><span class="p-num">5</span><span class="p-time">(11:00 - 11:30)</span></th>
        <th><span class="p-num">6</span><span class="p-time">(11:30 - 12:00)</span></th>
        <th><span class="p-num">7</span><span class="p-time">(12:00 - 12:25)</span></th>
        <th><span class="p-num">8</span><span class="p-time">(12:25 - 12:50)</span></th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="day in filteredSchedule" :key="day.dayIndex">
        <td class="day-col" :class="{'is-current-day': day.dayIndex === currentDayIndex}">
          {{ day.day }}
        </td>

        <ScheduleCell
          v-for="(classData, idx) in day.classes"
          :key="idx"
          :class-data="classData"
          :is-active="activePeriod?.dayIndex === day.dayIndex && activePeriod?.periodIndex
=== idx"
          :progress="activePeriod?.progressPercent"
        />
      </tr>
    </tbody>
  </table>
</template>
```

```
<script setup>
import { computed } from 'vue';
import ScheduleCell from './ScheduleCell.vue';

const props = defineProps({
  scheduleData: Array,
  isShowingAllDays: Boolean,
  currentDayIndex: Number,
  activePeriod: Object
});

// Computed property to automatically handle filtering
const filteredSchedule = computed(() => {
  if (props.isShowingAllDays) return props.scheduleData;
  // If showing today only, filter by current day (or Sunday if weekend)
  const targetDay = props.currentDayIndex > 4? 0 : props.currentDayIndex;
  return props.scheduleData.filter(d => d.dayIndex === targetDay);
});
</script>

<style scoped>.schedule-table {
  width: 100%;
  border-collapse: collapse; background: white;
  box-shadow: 0 10px 30px rgba(0,0,0,0.1); border: 2px solid var(--border-color);
}
th, td { border: 1px solid var(--border-color); text-align: center; padding: 12px 5px; width:
11%; }
th { background-color: var(--header-bg); color: white; font-size: 0.9rem; line-height:
1.4; }.p-num { font-size: 1.1rem; font-weight: bold; display: block; }.p-time { font-size:
0.75rem; color: #ccc; }.day-col {
  background-color: var(--day-bg); color: var(--day-text);
  font-size: 1.1rem; font-weight: bold; text-align: left;
  padding-left: 15px; width: 12%;
}.is-current-day { border-left: 5px solid var(--live-glow); }
</style>
```

---

4. Schedule Cell (src/components/ScheduleCell.vue)

By moving the cell into its own component, the complex logic regarding colors, NAFS labels, and the red progress line is entirely contained and isolated.

```
<template>
  <td class="subject-cell" :class="[colorClass, { 'is-current-class': isActive }]">

    <div v-if="isActive" class="cell-progress-fill" :style="{ height: progress + '%' }"></div>

    <div v-if="isActive" class="cell-progress-line" :style="{ top: progress + '%' }"></div>

    <span class="main-text">{{ classData.sub }}</span>
    <span v-if="classData.nafs" class="nafs-text">(NAFS M)</span>

  </td>
</template>

<script setup>
import { computed } from 'vue';

const props = defineProps({
  classData: Object,
  isActive: Boolean,
  progress: Number
});

// Automatically apply correct color classes
const colorClass = computed(() => {
  if (props.classData.sub === '7A') return 'sub-7A';
  if (props.classData.sub === '4A') return 'sub-4A';
  if (props.classData.sub!== '') return 'sub-other';
  return 'sub-free';
});
</script>

<style scoped>.subject-cell {
  font-size: 1.2rem; font-weight: 900; color: #111;
  position: relative; transition: all 0.3s; overflow: hidden;
}.nafs-text {
  display: block; font-size: 0.7rem; font-weight: normal;
  color: #444; margin-top: 5px; position: relative; z-index: 5;
}.main-text { position: relative; z-index: 5; }

/* Colors */.sub-7A { background-color: var(--7a-color); }.sub-4A { background-color:
var(--4a-color); }.sub-other { background-color: var(--other-color); }.sub-free
{ background-color: white; }
```

```
/* Active State */.is-current-class { box-shadow: inset 0 0 0 3px var(--live-glow), 0 0 15px rgba(255, 42, 42, 0.4); z-index: 10; }.cell-progress-line {
  position: absolute; left: 0; right: 0; height: 3px;
  background: rgba(255, 42, 42, 0.8); z-index: 1; pointer-events: none;
  box-shadow: 0 0 8px rgba(255, 42, 42, 0.6);
}.cell-progress-line::before {
  content: ''; position: absolute; left: 0; top: -4px; width: 10px; height: 10px;
  background: var(--live-glow); border-radius: 50%; box-shadow: 0 0 5px rgba(255, 42, 42, 0.8);
}.cell-progress-fill {
  position: absolute; top: 0; left: 0; right: 0;
  background: rgba(255, 42, 42, 0.1); z-index: 0; pointer-events: none;
}
</style>
```

Setup Note for Vue:

Make sure your full_schedule.json, icon.png, notification1.mp3, manifest.json, and sw.js are stored in your Vue project's **public/** folder so they can be fetched securely by the browser.

> "Make sure your full_schedule.json, icon.png, notification1.mp3, manifest.json, and sw.js are stored in your Vue project's public/ folder so they can be fetched securely by the browser." these files for the whole vue app or this page only.what if i want it to be for this page only.

That is a very smart question. By default, anything you put in the public/ folder belongs to the **entire Vue app**.

If you are adding this schedule to an existing Vue website (like a school portal or personal site) and you only want these files to apply to this specific "Schedule" page, we need to change our approach.

Here is exactly how to scope these files so they don't pollute your global app.

1. The Data and Audio (Move into src/)

In Vue, you don't need to use fetch() to read a local JSON file, and you don't need to put the audio in the public folder. You can treat them as **local component assets**.

Move full_schedule.json and notification1.mp3 into your src/components/ folder (or src/assets/), right next to your Vue files.

Then, update your Vue component to import them directly:

```
// In your Vue component script:
import scheduleDataRaw from './full_schedule.json'; // Imports directly! No fetch needed.
import alertSoundFile from './notification1.mp3'; // Webpack/Vite will handle the URL

const scheduleData = ref(scheduleDataRaw);
const alertSound = new Audio(alertSoundFile);
```

Why this is better: Webpack/Vite will now bundle these files specifically for this component. If you ever delete the component, the assets get cleaned up automatically.

2. The PWA Files (manifest.json & sw.js)

Service Workers and Manifests are a bit tricky because web browsers *expect* them to be standard URLs. You generally still have to put manifest.json, sw.js, and icon.png in the public/ folder, but **you can lock them down so they only control your schedule page.**

A. Scope the Manifest: Open your manifest.json and add a scope and start_url. If your schedule page lives at www.yoursite.com/schedule, do this:

```
{
  "name": "My Weekly Schedule",
  "short_name": "Schedule",
  "start_url": "/schedule/",
  "scope": "/schedule/",
  "display": "standalone",
  "icons": [ { "src": "/icon.png", "sizes": "192x192", "type": "image/png" } ]
}
```

The scope tells the phone: "Only act like an app when the user is on the /schedule/ page."

B. Scope the Service Worker: When you register the Service Worker in your Vue component, add a scope parameter. Change your registration code to look like this:

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    // The { scope } restricts the offline caching to ONLY this specific URL path
```

```
  navigator.serviceWorker.register('/sw.js', { scope: '/schedule/' }).then(registration => console.log('SW registered for schedule only')).catch(err => console.log('SW failed: ', err));
 });
}
```

C. Only load the manifest on this page: Instead of putting <link rel="manifest" href="/manifest.json"> in your global index.html (which would trigger the install prompt on your homepage), you should dynamically inject it only when the Schedule component loads.

Add this inside your onMounted hook in the Vue component:

```
onMounted(() => {
 // Inject manifest only for this page
 let link = document.createElement('link');
 link.rel = 'manifest';
 link.href = '/manifest.json';
 document.head.appendChild(link);
});
```

Summary of where things go:

- 📁 src/assets/full_schedule.json *(Imported in Vue)*
- 📁 src/assets/notification1.mp3 *(Imported in Vue)*
- 📁 public/manifest.json *(Scoped to /schedule/)*
- 📁 public/sw.js *(Registered with { scope: '/schedule/' })*
- 📁 public/icon.png

Would you like me to rewrite the App.vue script section to show you exactly how all of these imports and scoping rules look when put together?